
Desarrollo de aplicación para asistir en el
aprendizaje de las matemáticas
Development of an application to assist in
learning mathematics



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Jorge Polo Peyres

Director

Gonzalo Méndez Pozo

Pablo Gervás Gómez-Navarro

Grado en Ingeniería Informática

Facultad de Informática

Universidad Complutense de Madrid

Desarrollo de aplicación para asistir en el
aprendizaje de las matemáticas
Development of an application to assist in
learning mathematics

Trabajo de Fin de Grado en **Ingeniería Informática**

Autor

Jorge Polo Peyres

Director

Gonzalo Méndez Pozo

Pablo Gervás Gómez-Navarro

Convocatoria: *Junio 2026*

Grado en Ingeniería Informática

Facultad de Informática

Universidad Complutense de Madrid

25/05/2026

Resumen

Desarrollo de aplicación para asistir en el aprendizaje de las matemáticas

Muchos estudiantes tienen dificultades para comprender plenamente todos los conceptos matemáticos que se enseñan en la escuela. Una forma de apoyar la educación en este ámbito es reforzar el aprendizaje de los estudiantes mediante problemas. Con un diseño cuidadoso de cómo se presentan los ejercicios, junto con pistas o una guía paso a paso, se puede construir una herramienta de aprendizaje muy productiva.

En este proyecto desarrollamos una aplicación que aborda estos problemas ofreciendo recomendaciones personalizadas y un conjunto de problemas suficientemente amplio, que cubre tanto los temas vistos en clase como los conocimientos previos más sencillos necesarios para abordarlos. Para lograrlo, abordamos los tres principales retos que plantea la implementación de este tipo de plataforma:

1. ¿Cómo interactúan los estudiantes con una plataforma de este tipo, qué problemas suelen encontrarse al utilizarla y cómo podemos construir una interfaz de usuario adecuada, basada en investigaciones existentes?
2. ¿De dónde obtenemos los problemas matemáticos, cómo construimos la base de datos y cómo conseguimos que los problemas procedentes de distintas fuentes se ajusten al formato de la base de datos?
3. ¿Cómo podemos personalizar la experiencia de aprendizaje del usuario y cómo construiríamos un sistema de recomendación?

Este proyecto muestra el potencial de los Modelos de Lenguaje Grandes para la generación de conjuntos de problemas matemáticos. También implementa una aplicación responsive que ofrece una experiencia similar a la de un tutor mediante un sistema de recomendación personalizado, pistas y un chatbot que asiste durante la resolución de problemas.

Palabras clave

Aprendizaje de matemáticas, transformación de conjuntos de datos, diseño de interfaces de usuario, patrón Model-View-ViewModel, .NET MAUI, sistema de recomendación, Bayesian Knowledge Tracing.

Abstract

Development of an application to assist in learning mathematics

Many students struggle to fully understand all the concepts of math learned in school. One way of supporting the education in this area is to reinforce the students with problems. With careful design of how to deliver the exercises, along with hints or step-by-step guidance, a very productive learning tool can be built.

For this project, we develop an application that addresses these issues by offering personalized recommendations and a sufficiently broad problem set, covering both the topics seen in class and the simpler prerequisites needed to approach them. In order to achieve this, we will address the three main issues while implementing this kind of platform:

1. How do students interact with a platform of this type, what are the problems that they often encounter while using it, and how can we build a user interface appropriate for the user, informed by existing research?
2. Where do we find the math problems, how do we build the database, and how do we make the problems from different sources fit the database?
3. How can we personalize the user learning experience and how would we build a recommendation system?

This project shows the potential of Large Language Models for generating mathematical problem datasets. It also implements a responsive application that provides a tutor-like experience through a personalized recommendation system, hints, and a chatbot to assist during problem solving.

Keywords

Learning mathematics, dataset transformation, user interface design, Model-View-ViewModel pattern, .NET MAUI, recommendation system, Bayesian Knowledge Tracing.

Contents

1. Introduction	1
1.1. Motivation	1
1.2. Objectives	2
1.3. Document structure	2
2. State of the Art	3
2.1. Existing Datasets	3
2.1.1. Math dataset	3
2.1.2. GSM8K	3
2.1.3. AMPS dataset	4
2.1.4. Orca-Math	4
2.2. Large Language Models	4
2.2.1. DeepSeek-R1	4
2.2.2. Mathstral	5
2.3. Math Reasoning and Evaluation Tools	5
2.3.1. MARIO Eval	5
2.3.2. CoinMath	5
2.4. Other math tutors	6
2.4.1. Duolingo	6
2.4.2. Khan Academy	6
2.4.3. Brilliant.org	7
2.4.4. Prodigy Math	7
2.5. Existing Recommendation Systems	7
2.5.1. Common Architecture	7
2.5.2. YouTube	7
2.5.3. Netflix	8
2.6. Knowledge Tracing Approaches	8
2.6.1. Knowledge tracing	8
2.6.2. BKT	9
2.6.3. LSTM	9
2.6.4. DKT	9

3. Project Description	13
3.1. Technological choices	13
3.1.1. Application Layer: User Interface & Communication	13
3.1.2. Dataset Layer: Content and Quality Assurance	14
3.1.3. Recommendation System Layer: Personalization	16
3.2. Dataset layer: collecting and generating problems	17
3.2.1. Taxonomy and Knowledge Components	18
3.2.2. Base dataset	18
3.2.3. How to transform the dataset?	19
3.2.4. Discussion of Results	20
3.3. Recommendation system layer	20
3.3.1. Conceptual architecture	20
3.3.2. BKT Model	21
3.3.3. Heuristic Model	21
3.3.4. Softmax Sampling	24
3.3.5. Difficulty control	24
3.3.6. Considerations and limitations	25
3.4. Application layer	26
3.4.1. Backend	26
3.4.2. Frontend	28
3.4.3. Navigation example	30
4. Conclusions and Future Work	37
4.1. Conclusions	37
4.1.1. Dataset generation	37
4.1.2. Recommendation system	39
4.1.3. Application	39
4.1.4. Evaluation against objectives	40
4.2. Future Work	40
4.2.1. Better base dataset	40
4.2.2. Faster dataset generation	40
4.2.3. Engagement and gamification	40
4.2.4. Recommendation system improvements	40
4.2.5. Quantitative evaluation	41
4.2.6. Mobile deployment	41
4.2.7. Closing remarks	41
5. Introducción	43
5.1. Motivación	43
5.2. Objetivos	44
5.3. Estructura del documento	44
6. Conclusiones y Trabajo Futuro	45
6.1. Conclusiones	45

6.1.1.	Generación del conjunto de datos	45
6.1.2.	Sistema de recomendación	47
6.1.3.	Aplicación	47
6.1.4.	Evaluación frente a los objetivos	48
6.2.	Trabajo Futuro	48
6.2.1.	Mejor conjunto de datos base	48
6.2.2.	Generación de datos más rápida	48
6.2.3.	Compromiso y gamificación	49
6.2.4.	Mejoras del sistema de recomendación	49
6.2.5.	Evaluación cuantitativa	49
6.2.6.	Despliegue móvil	49
6.2.7.	Reflexiones finales	49

Bibliography	51
---------------------	-----------

List of figures

2.1. Recurrent neural network architecture used in DKT	9
2.2. RNN equations	10
2.3. LSTM equations	10
2.4. Loss function for DKT	10
2.5. Influence function J_{ij} definition	11
2.6. AUC comparison between BKT and LSTM on the Khan dataset . . .	11
2.7. Conditional influence graph on the Khan Academy dataset	12
3.1. Pydantic models for the POST <code>/sessions/{id}/attempt</code> endpoint. .	27
3.2. MVVM pattern as applied in the MAUI client. Views (XAML pages) bind to ViewModels (PageModel classes), which in turn invoke Services that handle HTTP communication and hold the data Model classes.	29
3.3. Navigation graph between MAUI pages. Solid arrows represent direct, contextual transitions (e.g., tapping a problem opens its detail page). Dashed arrows represent navigation through the app shell tab bar. . .	30
3.4. Main Page empty	31
3.5. Profile Page	31
3.6. Questionnaire Page	32
3.7. Main Page full	32
3.8. Problems Library Page	33
3.9. Problem Detail Page regular	33
3.10. Problem Detail Page hints	34
3.11. Problem Detail Page chatbot	34
3.12. Problem Detail Page answer	35
3.13. Settings Page	35

List of tables

2.1. Performance of math LLMs on common benchmarks.	5
3.1. Implementation and Alternatives for the Application Layer	13
3.1. Implementation and Alternatives for the Application Layer (Continued)	14
3.2. Implementation and Alternatives for the Dataset Layer	15
3.2. Implementation and Alternatives for the Dataset Layer (Continued) .	16
3.3. Implementation and Alternatives for the Recommendation System Layer	16
3.3. Implementation and Alternatives for the Recommendation System Layer (Continued)	17
3.4. MongoDB collections used by the backend.	19
3.5. BKT parameters	21
3.6. Scoring weights per user objective. w_I is interest, w_N is need, w_U is uncertainty, and w_D is diversity.	23
3.7. Fuzzy rules	25
3.8. REST endpoints exposed by the FastAPI backend, grouped by router.	27
4.1. Distribution of Knowledge Components in the final dataset of 777 problems.	38
4.2. Distribution of problems across Spanish secondary-school courses in the final dataset.	38
4.3. Distribution of difficulty levels in the final dataset.	39
6.1. Distribución de los Componentes de Conocimiento en el conjunto de datos final de 777 problemas.	46
6.2. Distribución de los problemas por cursos de la educación secundaria española en el conjunto de datos final.	46
6.3. Distribución de los niveles de dificultad en el conjunto de datos final.	47

Introduction

It is well known that education is one key aspect of society. One generation cannot progress more than the one before if they do not learn at least the most valuable lessons that were taught to the prior generation. Nowadays, new technology that uses Large Language Models will influence the way education works. During this project I tried to see which areas can be influenced by this, and to what extent.

1.1. Motivation

In the experience of many students, what is taught in class can fall short of their expectations, while in other cases the lessons may be too demanding. Regardless of which of these situations applies, the most effective solution is to provide some support to the student from a math tutor. This is the solution that has been proven to work in article Bloom (1984), where it discusses how students receiving one-on-one tutoring combined with mastery learning perform two standard deviations better than average students in a traditional classroom. However, this is not the most affordable option. With all the recent advances in technology and Artificial Intelligence, there should be an effective way to substitute the math tutor with a software application.

In fact there have been plenty of programs that aim to solve this issue, and we will be studying their implementation in Chapter 2. Nevertheless, I have not been able to find one that fully matches the way the official Spanish education system teaches mathematics, or at least supports it independently.

This project takes a layered approach, where each layer was built to be implemented and tested independently. The dataset layer consists of building the problem dataset and constructing the database infrastructure that will store the relevant interactions of the user with the application. The recommendation system layer is the intelligent part of the system, responsible for deciding which problems to present to each user. The application layer contains the cross-platform UI that the user interacts with.

1.2. Objectives

The objectives of this project are:

- Build a library of math problems labeled by topic, course, and difficulty, suitable for the Spanish secondary education curriculum.
- Design a backend system that exposes these problems through a REST API.
- Develop a cross-platform UI that engages students and supports the daily-session learning loop.
- Implement a recommendation system that personalizes problem selection based on user interactions and stated interests.
- Evaluate the resulting system against existing tutoring platforms.

The scope of this project is limited to the Spanish secondary education curriculum (ESO and Bachillerato). It does not address features for teachers (such as classroom dashboards or analytics), nor does it implement multi-user collaborative learning. Empirical evaluation with real students is also out of scope; the evaluation is restricted to a comparison against existing tutoring platforms.

1.3. Document structure

The remainder of this document is organized as follows. Chapter 2 reviews existing math datasets, large language models for math reasoning, prior math tutoring platforms, and the recommendation system literature. Chapter 3 describes the design and implementation of the system across its three layers: dataset, recommendation, and application. Chapter 4 discusses the conclusions drawn from the work and outlines directions for future improvement.

Chapter 2

State of the Art

This chapter focuses on the study of the most modern techniques or technologies that are used for the implementation of some aspect of the project. Some studies mentioned in this chapter were not implemented in the final version of the project, but they serve as inspiration for other parts or are used for comparison. The chapter is divided into different sections for each block of content that serve this project.

2.1. Existing Datasets

Many math datasets have been created for different tasks. Our problem requires a dataset with very accurate labels and a comprehensive step-by-step answer that can clear up the students' doubts.

2.1.1. Math dataset

The Math Hendrycks et al. (2021) dataset has 12,500 problems taken from math competitions. The architecture of the database provides for each problem the problem statement, the tag, the level, and the explained answer. It was used to test LLM performance in 2021, where models scored between 3% and 6.9% correct, while computer science PhD students scored 40%. These numbers may seem harsh on the models tested; however, more recent data show that current models score above 40%. This dataset has proven to have several usages, such as training LLMs. But if it were to be used for helping students who may be struggling at school, the high level of difficulty from the problems might make the issue worse rather than help the student.

2.1.2. GSM8K

GSM8K Cobbe et al. (2021) is a dataset with 8.5K math problems that provide the statement and a natural language answer. It focuses specifically on multi-step mathematical reasoning in the context of grade school word problems and it has been used to improve LLMs performance. The complexity is moderate, requiring between

2 to 8 steps to solve each problem. It used to be difficult for LLMs, DeepSeek-Math-7B-RL model achieved 58.8% of accuracy, while DeepSeek-R1-Distill-Llama-8B achieved 89.1%. In order to train LLMs to solve the dataset, two methods were used:

- **Baseline:** fine-tuning a generator model on the training set.
- **Verifiers:** training a separate model to rank the correctness of each solution, which simulates multi-step reasoning.

2.1.3. AMPS dataset

The AMPS dataset Hendrycks et al. (2021) was introduced in 2021 alongside MATH, with millions problems providing step-by-step solutions with the purpose of training LLMs to understand basic math principles. It is divided in two different parts:

- The Khan Academy Component: With approximately 100,000 problems that cover the standard K-12 math curriculum.
- The Mathematica Component: Has over 5 million generated problems using Mathematica software.

2.1.4. Orca-Math

Orca-Math Mitra et al. (2024) is a synthetic dataset of 200,000 problems, produced by using LLMs to generate new problems from existing datasets like GSM8K and to write a full step-by-step solution. The dataset provides the problem statement together with a step-by-step explanation of how to solve the problem, including the final answer. The objective of creating this dataset was to train Small Language Models (SLMs) to solve math problems at a higher level. A Mistral-7B model was trained on this dataset and achieved 86.81% accuracy on GSM8K, which was significantly better than Llama-2-70B, Gemini Pro and GPT-3.5.

2.2. Large Language Models

In order to make the best out of these datasets, we might need to transform their structure in order to fit the application format. We may also need the steps to reach the solution. One solution to these issues is to use LLMs as long as the solutions they provide are accurate.

2.2.1. DeepSeek-R1

DeepSeek-R1 DeepSeek-AI (2025) uses a Mixture-of-Experts (MoE) architecture. It has a massive total parameter count ($\sim 671\text{B}$), but only a small fraction ($\sim 37\text{B}$) is activated per token, making inference significantly faster than a fully dense model

of that size. It has a dedicated Reasoning Model that prioritizes detailed, step-by-step logic over fast, direct answers, which will prove to be very effective when solving mathematical problems. It has a very large context window of 128,000 tokens, allowing it to handle complex, multi-page problems or vast contexts for advanced reasoning. The model weights are released under the highly permissive MIT License, allowing us to use it. The performance of different models on common math benchmarks is shown in Table 2.1.

Benchmark	Dataset	DeepSeek-Math-7B-RL	DeepSeek-R1-Distill-Llama-8B	DeepSeek-R1 (671B)	OpenAI o1-1217
MATH	Math Competition (Hard)	58.8%	89.1%	97.3%	96.4%
GSM8K	Grade School Math	88.2%	89.6%	99.3%	99.1%
AIME 2024	Elite Math Olympiad	~10–15% (est.)	72.6%	79.8%	79.2%

Table 2.1: Performance of math LLMs on common benchmarks.

2.2.2. Mathstral

Mathstral Mistral AI (2024) is a model built on Mistral 7B transformer architecture, specialized on mathematical reasoning. It supports 32,000-token context window. The model is very efficient and can be used with a less powerful GPU, since it has 7B parameters. Its weights are released as open-source under the Apache 2.0 License.

2.3. Math Reasoning and Evaluation Tools

2.3.1. MARIO Eval

MARIO Eval Zhang et al. (2024) is a toolkit designed to leverage numerical accuracy and natural language comprehension in mathematical datasets. It addresses the lack of generalizability across math datasets, since each dataset tends to design its own evaluation script, and helps prevent inconsistencies. It uses a hybrid approach built with a Python Computer Algebra System (CAS) for numerical precision and an optional LLM for the evaluation. This toolkit achieves an improved evaluation for math datasets.

2.3.2. CoinMath

CoinMath Wei et al. (2025) studies different ways of improving LLMs’ math performance. One way is to use concise comments, which helps the model align the code structure with the natural language reasoning steps. Another way is to use descriptive naming for variables and functions, which shows an enhancement on reasoning. Finally, including hard-coded solutions improves performance. These techniques showed a significant improvement from the baseline model, MAMmoTH, which was the state-of-the-art Math LLM at that time.

2.4. Other math tutors

2.4.1. Duolingo

The Duolingo application uses a combination of diagnostic engines, business rules, heuristic models for selecting items and a lot of product engineering as they describe in their Duolingo Papers Freeman et al. (2023). The Duolingo learning method consists of five principles:

1. **Learn by doing:** you don't need an introduction to start solving problems. Students learn through repetition and pattern acquisition, formally studied in the field of statistical learning Schapiro y Turk-Browne (2015).
2. **Learn in a personalized way:** using machine-learning-based models (Bird-brain) trained on millions of problems solved daily, the system estimates how proficient the student is and how difficult the problems are. The model focuses on the difficulty range appropriate for the student's learning.
3. **Focus on what matters:** the courses are designed based on the Common European Framework of Reference (CEFR).
4. **Stay motivated:** the hardest part of learning is staying consistent. Duolingo's solution is to use gamification techniques such as win streaks.
5. **Feel the delight:** they use characters and stories that make learning more fun. Additionally, various visual tools for problem solving help make learning more entertaining.

Duolingo uses a system called Birdbrain Santosh (2024) that uses several architectures. Its main model is based on a Long Short-Term Memory (LSTM) Sherstinsky (2020), which predicts student performance based on their knowledge status, while also being able to analyze sequential data over time effectively. It also uses a statistical model to choose when to review previously viewed material. Moreover, it uses a different model that uses half-life regression, which identifies concepts that are often forgotten over time using the error patterns of millions of users. Another model that is used is an adaptive review scheduling to choose where and when to insert these problems to maximize user retention. Finally, Duolingo uses a bandit algorithm for its notification system.

2.4.2. Khan Academy

Khan Academy Khan Academy (2026a) is a non-profit that offers a massive library of courses and it has a Course Mastery feature which works as a recommendation system, which works by giving the user a Course Challenge that evaluates the ability of the student and adjusts the recommendation accordingly.

It also integrates a generative AI tutor called Khanmigo Khan Academy (2026b), that works by helping students get to the answer rather than giving it right away.

2.4.3. Brilliant.org

Brilliant Brilliant.org (2026) is a math and science learning platform that works with interactive problems. It uses an adaptive learning system that provides personalized recommendations to the users.

2.4.4. Prodigy Math

Prodigy Math is a platform focused on students that have 1-8th grade math Prodigy Education (2026). The gamification of this app outlines very well the core of adaptive learning, using an AI that starts with an initial diagnostic for a student and adjust as the student progresses.

2.5. Existing Recommendation Systems

The main objective of a recommendation system is to reduce a universe of 10^8 items to one of 10^2 items.

2.5.1. Common Architecture

The usual large-scale recommendation systems include: Candidate Generation (Retrieval), Scoring (Ranking) and sometimes Re-ranking.

- **Candidate Generation (Retrieval):** Starting from a huge size of items the recall stage narrows it down to a few hundred or thousand candidates. Most systems implement this via embeddings. The probability of recommending video i to user u can be modeled as a softmax over all videos. The retrieving of the nearest items to a user's embedding is done by using approximate nearest neighbor (ANN) algorithm.
- **Scoring (Ranking):** This stage takes the candidates and predicts which items the user is most likely to engage with. It then outputs a score for each item. This stage can afford a more expensive model because it runs only on hundreds of candidates.
- **Re-ranking:** This stage might ensure that some metrics are met for the candidates such as variety. It can also down-rank items that the user has seen before.

2.5.2. YouTube

YouTube's recommendation system Covington et al. (2016) runs using two deep neural networks, one is a **deep candidate generation model** and the other one is a **deep ranking model**. Both are feed-forward neural networks trained on massive datasets. The candidate generation is essentially an encoder that produces a compact user representation and then retrieves videos that might interest the user

using a softmax output. The ranking network evaluates those candidates with many additional features to predict an engagement score.

The features of the videos, such as the feedback or video IDs, are embedded into dense vectors so that the model can learn similarities between different videos.

The ranking is trained by using the user’s clicks on videos and watch retention. The loss function is weighted such that videos that were shown but not clicked count as negative examples.

These models are massive with around 1 billion parameters.

2.5.3. Netflix

Netflix’s recommendation system Gomez-Urbe y Hunt (2015) prioritizes finding what users want to watch quickly, since their data found that after 60-90 seconds of browsing for what to watch the users tend to close the app. They also found that about 80% of the contents watched comes from recommendations in the system, not from direct searches.

Netflix’s homepage contains rows containing items with different themes. Both the rows and the items are personalized to the user. How it works:

- **Personalized video ranker (PVR):** Ranks all items in the catalog for a user to decide the order of items in a given row or category.
- **Top-N ranker:** Select the most relevant items globally.
- **Continue watching:** Considers variables such as the last time you watched the item or how much you’ve watched it to decide the order of appearance.
- **Because you watched “...”:** Chooses the most similar items to the base item X . Out of that list the order is personalized to the user.
- **Page generation:** This algorithm selects a subset of the available rows that will cover best the spectrum of things the user might want to watch and arrange them in an effective order.

Under the hood, it uses a neural network for the ranking. But another approach that helps the system is the use of altgenres created by Netflix. These micro-genres are a result of clustering content by attributes and users viewing patterns and help to group similar content.

The recommendations are usually generated when log in occurs or daily.

2.6. Knowledge Tracing Approaches

2.6.1. Knowledge tracing

Given a set of interactions of a student’s outcomes $x_0, \dots, x_t$, having for each interaction a tuple with the question tag and whether the student’s answer was correct or incorrect: $x_t = \{q_t, a_t\}$. What we want to do is predict aspects of the

next interaction x_{t+1} , such as whether the answer would be correct depending on the type of question asked. This task is called Knowledge tracing and there are several approaches for it.

2.6.2. BKT

Bayesian Knowledge Tracing Corbett y Anderson (1995) is the most common approach to student learning models. Knowledge is transformed into binary variables using a Hidden Markov Model (HMM), whose probabilities are updated as the student attempts problems. There are techniques to refine these concepts, such as Cognitive Task Analysis Schraagen et al. (2000), which uses expert protocols when solving problems to broaden the scope of information captured by the binary response.

2.6.3. LSTM

Long Short-Term Memory Sherstinsky (2020) is a type of Recurrent Neural Network (RNN) used to handle long-term dependencies. It uses three types of gates: Forget Gate (decides what information to discard from each Cell State), Input Gate (decides what part of the new information to store in the Cell State), and Output Gate (chooses what part of the Cell State to move to the Hidden State). The Cell State represents the main memory stream, and the Hidden State represents the short-term memory stream.

2.6.4. DKT

Deep Knowledge Tracing Piech et al. (2015) applies a combination of two types of RNNs: a vanilla RNN with sigmoid units and an LSTM model. The connection of variables in an RNN goes from X (input) through H (hidden state) and ends in Y (output), as shown in Figure 2.1.

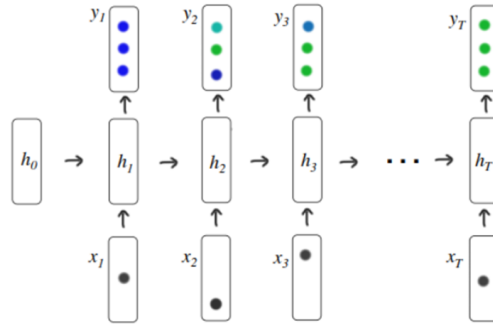


Figure 2.1: Recurrent neural network architecture used in DKT

The RNN update equations are shown in Figure 2.2, where $\sigma(\cdot)$ is the sigmoid function:

$$\begin{aligned}\mathbf{h}_t &= \tanh(\mathbf{W}_{hx}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h), \\ \mathbf{y}_t &= \sigma(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y),\end{aligned}$$

Figure 2.2: RNN equations

The LSTM equations are shown in Figure 2.3:

$$\begin{aligned}\mathbf{i}_t &= \sigma(\mathbf{W}_{ix}\mathbf{x}_t + \mathbf{W}_{ih}\mathbf{h}_{t-1} + \mathbf{b}_i) \\ \mathbf{g}_t &= \sigma(\mathbf{W}_{gx}\mathbf{x}_t + \mathbf{W}_{gh}\mathbf{h}_{t-1} + \mathbf{b}_g) \\ \mathbf{f}_t &= \sigma(\mathbf{W}_{fx}\mathbf{x}_t + \mathbf{W}_{fh}\mathbf{h}_{t-1} + \mathbf{b}_f) \\ \mathbf{o}_t &= \sigma(\mathbf{W}_{ox}\mathbf{x}_t + \mathbf{W}_{oh}\mathbf{h}_{t-1} + \mathbf{b}_o) \\ \mathbf{h}_t &= \mathbf{o}_t \odot \mathbf{m}_t \\ \mathbf{m}_t &= \mathbf{f}_t \odot \mathbf{m}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t \\ \mathbf{z}_t &= \mathbf{W}_{zm}\mathbf{m}_t + \mathbf{b}_z \\ \mathbf{y}_t &= \sigma(\mathbf{z}_t)\end{aligned}$$

Figure 2.3: LSTM equations

To convert these interaction into input vectors of fixed length we have two methods depending on the number of M problems:

- Low M : Set x_t to be the one-hot encoding of the tuple $h_t = \{q_t, a_t\}$, so $x_t \in \{0, 1\}^{2M}$.
- High M : Assign a random vector $n_{q,a} \sim \mathcal{N}(0, I)$ to each input tuple and set each $x_t = n_{(q_t, a_t)}$.

The output y_t is a vector of length $= M$ (number of problems) and represents the predicted probability of the student answering correctly the problem. Therefore, the prediction a_{t+1} can be obtained from the entry in y_t corresponding to q_{t+1} .

The training objective is the negative log likelihood of the students responses. The loss for a single student is defined by the function shown in Figure 2.4:

$$L = \sum_t \ell(\mathbf{y}_i^T \delta(q_{t+1}), a_{t+1})$$

Figure 2.4: Loss function for DKT

$\delta(q_{t+1})$ will be the one-hot encoding from the exercise answered in $t + 1$ and $\ell(\cdot)$ is the binary cross entropy, $\ell(y, \cdot)$ is the loss of one prediction.

The objective is minimized using stochastic gradient descent on minibatches. A dropout is applied to h_t when computing y_t and not when computing the next

hidden state to avoid overfitting. The models used a hidden dimensionality of 200 and a mini-batch size of 100.

To understand the relationship between the problems an influence function (J_{ij}) is defined for each pair of exercises and a synthetic dataset will be created so that the model understand the structure of skills or problem types. This can be seen in Figure 2.5:

$$J_{ij} = \frac{y(j|i)}{\sum_k y(j|k)},$$

Figure 2.5: Influence function J_{ij} definition

The results show an AUC of 0.68 for BKT on the Khan dataset, while the LSTM neural network reached an AUC of 0.85, as shown in Figure 2.6.

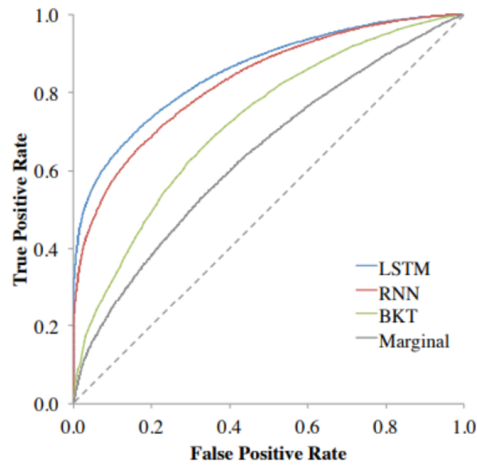


Figure 2.6: AUC comparison between BKT and LSTM on the Khan dataset

The results regarding the structure of the problems can be visualized in a graph of conditional influences where a directed edge from exercise A to exercise B shows how much a change in performance on A influences the predicted performance on B .

Figure 2.7 shows the graph produced by applying this method to the Khan Academy dataset, using only pairs (A, B) of exercises where B appears more than 1% of times after A .

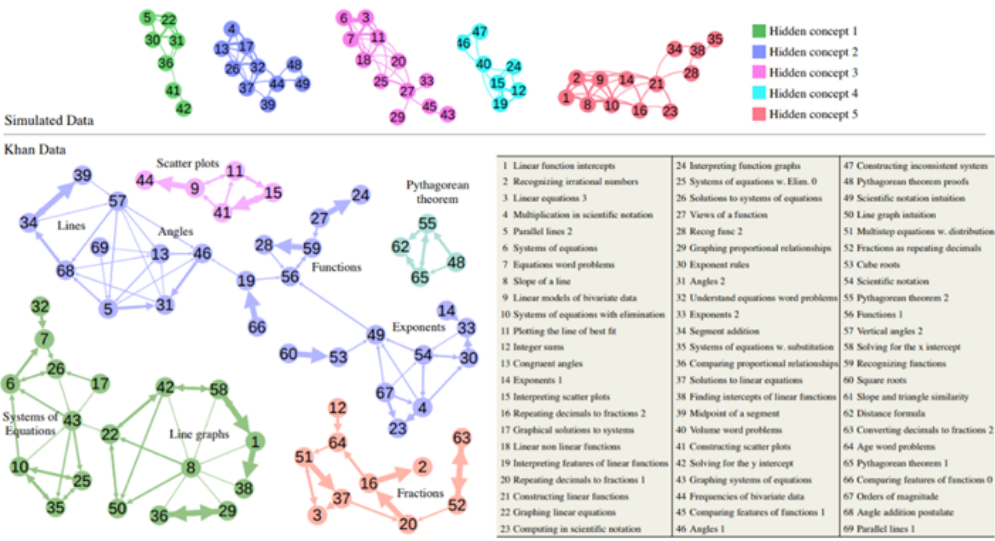


Figure 2.7: Conditional influence graph on the Khan Academy dataset

Project Description

This chapter describes how each part of the project was implemented. It begins with a section that analyzes the technological options considered and justifies the final choices, as well as the different techniques studied. The project gets divided into three layers, which work together, but were implemented separately and on different timelines.

3.1. Technological choices

3.1.1. Application Layer: User Interface & Communication

Table 3.1 summarizes the technologies chosen for the application layer, along with the alternatives that were considered and discarded.

Table 3.1: Implementation and Alternatives for the Application Layer

Component	Chosen Implementation	Alternative Implementations
Frontend UI Framework	.NET MAUI (C#, XAML) with the MVVM pattern. Single codebase that compiles to Windows, Android and iOS.	Flutter (Dart): excellent for highly animated UIs and fast Hot Reload. React Native (JavaScript/TypeScript): large community and ecosystem. Kotlin Multiplatform (KMP): shares business logic, but UI requires platform-specific layers.
Answer Input	MAUI Entry/Editor controls for numerical and short symbolic answers.	Custom Math Keyboard Component : dedicated MAUI control for complex input (fractions, matrices), simplifying server-side parsing but requiring extra UI engineering.

Continued on next page

Table 3.1: Implementation and Alternatives for the Application Layer (Continued)

Component	Chosen Implementation	Alternative Implementations
Back-end Framework	Python with FastAPI exposing RESTful endpoints. Python was required to call local LLMs through Ollama for both dataset generation and the chatbot.	Flask (Python): simpler but less ergonomic for typed request/response models. .NET Core Web API (C#): full technology-stack uniformity. Node.js (Express/Koa): scalable JavaScript option with no native LLM integration.
Local LLM Runtime	Ollama , used to run Mathstral (dataset generation) and DeepSeek-R1 (chatbot) locally on the same machine as the backend.	Hosted API (OpenAI, Anthropic, Mistral La Plateforme): faster inference but paid per token and dependent on an internet connection.
Communication Protocol	RESTful API via Http-Client (MAUI) exchanging JSON messages.	gRPC : high-performance, low-latency protocol, but higher setup complexity. SignalR / Web-Sockets : useful for real-time features, but overkill for a session-based learning app.

MAUI was the most suitable choice for a cross-platform application, and because the rest of the project also relies on it, communication with the backend was straightforward through HTTP endpoints. The backend itself had to be written in Python in order to call Ollama at runtime for both dataset generation and the chatbot. FastAPI was chosen over Flask because of its built-in support for Pydantic request and response models and its automatic OpenAPI documentation.

3.1.2. Dataset Layer: Content and Quality Assurance

Table 3.2 shows the different approaches possible for the dataset layer, highlighting the ones that were chosen.

Table 3.2: Implementation and Alternatives for the Dataset Layer

Component	Chosen Implementation	Alternative Implementations
Base Dataset	GSM8K as the source of problem statements, chosen for its size, structured format and free availability.	Bespoke Parametric Templates aligned with the Spanish curriculum: stronger curricular alignment but very labour-intensive to build from scratch. MATH dataset : broader topic coverage, but problem difficulty far exceeds the target audience.
Problem Generation	Three-phase LLM pipeline using Mathstral via Ollama: Phase A tags each problem with course, KCs and difficulty; Phase B normalises and translates the statement to Spanish; Phase C generates solution steps.	Direct LLM generation from scratch : prompting a large model for new problems, requires heavy human filtering. Fine-tuning a smaller LLM on Spanish curriculum data, more accurate but expensive to set up.
Generation Model	Mathstral (7B), chosen for the balance between quality and inference time. Avoided over-reasoning on ambiguous statements.	DeepSeek-R1 : discarded due to long inference times caused by its reasoning module and frequent overthinking on simple problems. GPT-4o / hosted models : higher quality but paid and not self-hosted.
Validation	Dedicated validation phase between Phase B and Phase C, where the LLM checks if the assigned tags actually fit the problem. Problems failing this check are flagged for future iteration.	MARIO Eval toolkit : hybrid Python CAS + LLM checker for standardised correctness validation. Manual review : accurate but does not scale beyond a few hundred problems.
Taxonomy and KCs	Custom taxonomy and Knowledge Components built via a RAG pipeline over Spanish curriculum documents, covering ESO and Bachillerato.	Adopting an existing taxonomy (e.g., Khan Academy’s): faster to integrate but not aligned with the Spanish curriculum.

Continued on next page

Table 3.2: Implementation and Alternatives for the Dataset Layer (Continued)

Component	Chosen Implementation	Alternative Implementations
Data Storage	MongoDB: schemaless document store for problems, users, sessions and attempts. JSON-native queries simplify dataset distribution analysis.	Relational database (PostgreSQL/MySQL): efficient for structured logs, but less flexible for nested solution content. PostgreSQL with JSONB: compromise option, document-like flexibility within a relational engine.

The final dataset is generated from GSM8K through a three-phase LLM pipeline driven by Mathstral, with a validation step between phases B and C. Mathstral was preferred over DeepSeek-R1 after testing, since the latter’s reasoning module added significant inference time without measurable quality improvements on simple grade-school problems. MongoDB was selected for its schemaless model and JSON-native queries, which made it easy to inspect the dataset distribution after each generation run.

3.1.3. Recommendation System Layer: Personalization

Table 3.3 represents the different techniques studied for a recommendation system and the technologies needed. The first column shows the one implemented in the project, while the second shows the alternatives that were discarded.

Table 3.3: Implementation and Alternatives for the Recommendation System Layer

Component	Chosen Implementation	Alternative Implementations
Knowledge Estimation	Bayesian Knowledge Tracing (BKT) per Knowledge Component: a 2-state Hidden Markov Model that estimates $P(L_{kc})$, the probability the student has mastered each KC.	Deep Knowledge Tracing (DKT): LSTM-based predictor, more expressive but requires a large interaction log and a training phase. Discarded due to lack of training data. Elo / IRT: simpler skill rating models but less direct for per-KC mastery.
Scoring Policy	Weighted heuristic combining interest, learning need, uncertainty and diversity, with weights determined by the user’s learning objective.	Pure BKT-driven selection: pick problems where mastery is between 60–80%. Ignores user preference and session diversity. Multi-Armed Bandit (UCB / ϵ-greedy): explicit exploration-exploitation trade-off, more complex to tune.

Continued on next page

Table 3.3: Implementation and Alternatives for the Recommendation System Layer (Continued)

Component	Chosen Implementation	Alternative Implementations
Sampling Policy	Softmax sampling with temperature $T = 0.35$, introducing controlled randomness so that identical profiles do not produce identical sessions.	Greedy selection: always pick the top-scoring KC, deterministic but repetitive. Pure random sampling: maximum diversity but ignores the scoring entirely.
Difficulty Control	Fuzzy logic controller based on accuracy and speed, treated as a PID-like feedback loop. Triangular membership functions for accuracy and trapezoidal for speed.	Fixed difficulty per user, set once via the questionnaire: simple but non-adaptive. Hard if-else rules: explicit and predictable but brittle at boundary cases.
Cold Start	Questionnaire-seeded priors: the user rates each KC and selects a target difficulty, which is mapped to initial $P(L_0)$ values.	Uniform priors: simpler but requires many sessions before recommendations feel personalised. Pre-trained DKT model fine-tuned on the user’s logs: more powerful but requires public interaction data.
Implementation	Pure Python logic integrated into the FastAPI backend. BKT updates and heuristic scoring run inline within request handlers.	Dedicated C++ library wrapper: higher performance if latency becomes an issue. Microservice architecture: decoupled recommender service, useful at scale but unnecessary for a single-user MVP.

The final recommendation system uses a hybrid approach that combines BKT for per-KC mastery estimation, a weighted heuristic scoring policy, softmax sampling for controlled randomness, and a fuzzy logic controller for problem difficulty. DKT was discarded because it requires a costly training phase and large interaction logs, neither of which were available at this stage of the project. The combined model is implemented entirely in Python and integrated directly into the FastAPI backend, with no external services or dependencies beyond MongoDB and Ollama.

3.2. Dataset layer: collecting and generating problems

The strategy here is to select a dataset as good as possible to use as basis. As expected, none of the publicly available datasets is good enough on its own, since

they often lack relevant features such as tags or topics.

Due to this, after selecting an appropriate dataset as our base, the next step is to build a better dataset from it. We cover how to achieve this below.

3.2.1. Taxonomy and Knowledge Components

The main issue with existing datasets is that the tags are not fit for the Spanish educational curriculum for mathematics. For this reason, we need to make sure that the new tags generated from the LLM are adequate and contained in a set of determined available knowledge components and more specific tags. The Taxonomy consists in a detailed description of the contents seen in each course, while the Knowledge Components are a small set of words that describe a topic or group of topics. For the sake of a better classification, besides the Knowledge Components we used Tags, which are more specific Knowledge Components. The Taxonomy is what gives the LLM the necessary information to classify each problem, while the KC/tags represent the domain of choices for the LLM.

3.2.1.1. Taxonomy

The Taxonomy follows the standard Spanish study plan from first grade of ESO up to second grade of Bachillerato. This taxonomy was built using a simple RAG architecture with mathstral and providing documentation from `matematicasinclusivas.com`, where the second Bachillerato curriculum is detailed, and `recursos.edu.xunta.gal`, where the secondary math textbook outlines can be found.

3.2.1.2. Knowledge Components

The Knowledge Component and Tags for each topic are built in JSON format using a similar method as the Taxonomy and providing the same documents. However, the prompt given to mathstral this time asks for a list of Knowledge Components that represent the different topics across all courses and Tags inside each different Knowledge Components that represent more specific areas of a topic and allow repetition across the different KCs. One example of a Knowledge Component is **Numbers** and one Tag inside this KC is **ExponentsAndRoots**.

3.2.2. Base dataset

I have decided to use GSM8K as the starting dataset because it is the one that balances these important principles for a dataset of this kind.

- **Quantity:** An app that tries to give a personalized experience needs to have enough items to choose from. Therefore, it is strictly necessary to achieve a sufficient amount of problems from each topic.
- **Structure:** For the sake of the app itself, the dataset will need to provide the problem statement and answer in good form.

3.2.3. How to transform the dataset?

With the method of construction that we are using for our final dataset of problems, we will essentially only need the problem statement, since the rest will be automatically generated.

This is due to the fact that we are using Mathstral, which is a Large Language Model, to read from GSM8K and generate the additional information needed for our platform. This process was divided into three phases so the execution could be more efficient, since each phase requires a different prompt and produces different amounts of information. The model finally chosen for this task was Mathstral after testing with DeepSeek-R1, although this LLM was discarded due to the long processing times caused by the reasoning module. Mathstral is the model that balances both performance and cost the best, which led me to decide to use it for this task. The phases in order of execution are:

- **Phase A: tagging + course + difficulty only:** This phase receives the defined taxonomy and the dictionary with Knowledge Components (KC) and tags available to use.
- **Phase B: statement normalization + final answer:** This phase normalizes the problem statement and final answer, translating them into Spanish.
- **Validation phase:** During this phase, problems will be reviewed to check if the tags fit the problem, so if they do not they will be marked as invalidated and in a future iteration it is possible to correct them.
- **Phase C: solution steps:** After the validation phase succeeds, this phase will generate logical steps to get to the desired solution, available gradually for users in the application.

The final database consists of four collections that get updated as the user practices on some problems, as shown in Table 3.4. The problems collection always allows new problems to be added without any modifications to the code.

Collection	Stored data
users	Profile, questionnaire answers, course, objective archetype.
users.bkt_state	Per-user BKT probabilities $P(L_{kc})$ for each KC. This is an Object inside of each user document in the users collection.
problems	Problem statement, answer, tags, KCs, course, difficulty, solution steps
sessions	Active and historical sessions, problem lists, timestamps
attempts	Individual attempt records: answer, correct flag, time spent

Table 3.4: MongoDB collections used by the backend.

3.2.4. Discussion of Results

At this stage, 799 problems have been processed with 777 problems stored in the database and 707 problems that have passed the validation stage and have been successfully processed for phase C. The problems that were not stored in the database had format issues in the original dataset. On the other hand, the ones that did not pass the validation stage might have ambiguous problem statements, incorrect answers or issues at phases A and B. These last issues are dependent on the quality of the base dataset.

3.3. Recommendation system layer

There are plenty of different approaches to recommendation systems. The well known ones that I have studied use complex structures such as multiple Neural Networks or DKT.

For this project I have decided to use a hybrid model that combines a Heuristic model and a Hidden Markov Model based on BKT. The BKT estimates knowledge and the heuristics decide what to do with that knowledge.

The main objective of a good recommendation system should be balancing these four metrics:

- **Preference:** The user receives problems from the tags and difficulty he wants.
- **Need:** The system tries to fix recurrent mistakes or weak topics.
- **Exploration:** The system should try to acquire recent performance data of all topics.
- **Session shape:** Each session should be coherent and have a natural flow.

3.3.1. Conceptual architecture

Essentially, the system is solving: *Given a student with a partial known knowledge across different KCs, what is the next optimal problem to maximize learning?*

The system decomposes into:

1. **State estimation (BKT):** $P(L_{kc})$ = probability of the student knowing a Knowledge Component.
2. **Heuristic policy (scoring):** $\text{score}(kc) = f(\text{interest, need, uncertainty, diversity})$
3. **Sampling policy (softmax):** The probability $P(kc)$ is proportional to $e^{\log(\text{score})/T}$.
4. **Shaping constraints:**
 - Number of problems.
 - Repetition/diversity penalties.
 - Difficulty controller

3.3.2. BKT Model

A Bayesian Knowledge Tracing model is able to predict somewhat accurately whether the user is actually learning or not. It is capable of selecting the recommended problems in such a way that the knowledge acquired is optimal.

In reality, BKT is a Hidden Markov Model (HMM) in disguise. For our problem, each KC is modeled as a 2-state HMM, where:

- $L=0$ means do not know.
- $L=1$ means knows.

This table shows the BKT variable parameters for each KC:

Parameter	Symbol	Meaning (Probability of)	Initial value
Initial knowledge	$P(L_0)$	Student knows the KC already	0.20
Learning rate	$P(T)$	Learning KC after one attempt	0.15
Slip	$P(S)$	Answer incorrectly despite knowing	0.10
Guess	$P(G)$	Answer correctly without knowing	0.20

Table 3.5: BKT parameters

These values have achieved a good balance for understanding when a student masters a Knowledge Component. Additionally, we set the mastery threshold to 0.90, meaning that a KC is considered mastered if $P(L_{kc}) \geq 0.90$.

The value that we are mostly interested in is $P(L)$, since this gives us whether the user knows a KC or not.

After one attempt, the parameters will update using these formulas: **Evidence update:**

$$P(L \mid \text{correct}) = \frac{P(L) \cdot (1 - P(S))}{P(L) \cdot (1 - P(S)) + (1 - P(L)) \cdot P(G)} \quad (3.1)$$

$$P(L \mid \text{incorrect}) = \frac{P(L) \cdot P(S)}{P(L) \cdot P(S) + (1 - P(L)) \cdot (1 - P(G))} \quad (3.2)$$

Learning update: (even if they failed, they might have learned)

$$P(L_{n+1}) = P(L \mid \text{evidence}) + (1 - P(L \mid \text{evidence})) \cdot P(T) \quad (3.3)$$

At the beginning, the system uses a cold start trick that consists of mapping the user preferences as prior knowledge. In other words, since we do not know how much a user knows at the start, we use the scoring for each kc that they provided.

3.3.3. Heuristic Model

In order to get an idea of what the user wants in terms of problem, the user will be able to do a questionnaire that establishes the initial weights for some aspects of the recommendation system. The questionnaire gets values for:

- Scores for each Knowledge Component (KC) or main topic from 1-5.

- Target difficulty.
- Cross-course policy: allow to recommend problems from above or below courses depending on performance, encouraging forward courses more than lower courses.
- Objective-dependent behavior.

3.3.3.1. Scoring

Initially, the scoring system for each KC is obtained from this formula:

$$\text{score}(\text{kc}) = w_I I_{\text{kc}} + w_N N_{\text{kc}} + w_U U_{\text{kc}} + w_D D_{\text{kc}} - P_{\text{recent}} - P_{\text{repeat}} \quad (3.4)$$

Where:

- I_{kc} : normalized questionnaire interest from user preferences.
- N_{kc} : learning need from BKT when available, else from accuracy.
 - Without BKT: $N = 1 - \text{accuracy}$
 - With BKT: $N = 1 - P(L_{\text{kc}})$
- U_{kc} : uncertainty: high when attempts are few. $U = 1/(1 + \text{attempts}/p_U)$
- D_{kc} : diversity bonus: higher if a KC is not in the current session. $D = 1/(1 + \text{session_count})$
- $w_I \dots w_D$: weights determined by the user's primary objective.
- P_{recent} and P_{repeat} : penalties from session-level dynamics.

3.3.3.2. Session-Level Dynamics

When generating each session, the backend tracks running counts of how many times each Knowledge Component, tag, difficulty level and course have been selected so far in the session being built. These running counts feed two complementary penalty terms in the scoring formula, P_{recent} and P_{repeat} .

The **recent-history penalty** P_{recent} looks at the last few items from the user's past sessions (the default window is the last four attempts). For each KC, tag, difficulty or course that already appears in this window, a fixed multiplier is subtracted from its score. This discourages the system from re-recommending the same items the user has just been working on, even if they would still score highly on the heuristic.

The **within-session penalty** P_{repeat} uses the running counts of the session currently being generated. Each repetition adds a small fixed cost (around 0.15 per repetition) up to a soft cap, the *repetition limit per KC/tag* (default: 2). Once this cap is reached, the penalty becomes steeper (around 0.45 per additional repetition), which strongly discourages the same KC or tag from appearing more than two or three times in a single session of 5–10 problems.

The combination of these two penalties produces sessions that feel coherent rather than repetitive: a KC or tag may appear more than once when it has a high underlying score, but never to the point of dominating the session, and items that were heavily practiced in the previous session are less prioritized in favor of fresher content.

3.3.3.3. Objective-dependent behavior

The **objective-dependent behavior** is classified in four different archetypes:

- **PRACTICE**: will keep balanced weights
- **IMPROVE_GRADES**: prioritizes weak KCs and KCs from their interests, allowing more repetition.
- **REVIEW**: seeks more diversity across the topics with less exploration.
- **LEARN_NEW_CONTENT**: boosts unseen KCs and allows more forward difficulty or course movement.

The weights for each different objective are shown in Table 3.6.

Objective	w_I	w_N	w_U	w_D
PRACTICE	0.30	0.35	0.20	0.15
IMPROVE_GRADES	0.15	0.55	0.20	0.10
REVIEW	0.30	0.25	0.10	0.35
LEARN_NEW_CONTENT	0.20	0.15	0.45	0.20

Table 3.6: Scoring weights per user objective. w_I is interest, w_N is need, w_U is uncertainty, and w_D is diversity.

3.3.3.4. Cross-course policy

The cross-course policy allows the recommender to include problems from the course immediately above or below the user’s declared course level, instead of restricting the session to a single course. It is controlled by a flag set in the user questionnaire, and when disabled the user only receives problems from their selected course.

When enabled, the course selection uses fuzzy memberships over the user’s overall accuracy — the same triangular and trapezoidal functions described in the difficulty controller — to compute three soft levels: **low**, **medium** and **high** performance. These memberships are then mixed with the user’s primary objective to produce a weighted distribution over the current course and its two neighbours.

The weights are constructed as follows. The current course always has a base weight of 1.0. The previous course receives a small baseline (around 0.05) plus a much larger contribution from the *low* membership and a small contribution from the *medium* membership, so a user struggling with the current material is more

likely to be offered review problems from one course below. The next course receives a baseline of 0.20 plus contributions from the *high* and *medium* memberships, so a strong user is offered forward-course problems instead. If the user's primary objective is `LEARN_NEW_CONTENT`, the next-course weight is boosted further.

The resulting policy is therefore not a hard cutoff but a smooth blend. Concretely:

- With **weak performance**, the system selects mostly the current course, with a low chance of the previous course.
- With **medium performance**, the system stays mostly on the current course but allows a low probability of either neighbour.
- With **strong performance**, the system selects between the current and next course, with a higher chance of advancing if the user's objective is to learn new content.

After the weights are computed, the same recent-history and within-session penalties described above are applied to the course choice, so that even if forward-course problems become highly likely the session will not consist entirely of them.

3.3.4. Softmax Sampling

A KC is selected using:

$$P(\text{kc}) \propto \exp\left(\frac{\log(\text{score})}{T}\right) \quad (3.5)$$

Where T = temperature

I have chosen to use softmax in order to introduce controlled randomness that does not overfit the top KC. A low value for T will be more greedy, while a high value will be more exploratory. Currently we use $T = 0.35$ to stay focused on the top-scoring KC. It is important that we have some level of randomness to the recommendation system, so that two identical students may experience different learning paths.

3.3.5. Difficulty control

For selecting the difficulty we represent the user performance in two variables: **accuracy** and **speed**. Using these variables we create a set of rules that represent the desired logic.

The formula looks like this: $d^* = d_{\text{profile}} + \Delta_{\text{fuzzy}}$ Where:

- d_{profile} is the target difficulty selected by the user in the questionnaire.
- Δ_{fuzzy} is a value derived from accuracy and speed.

The variables can take these different values:

- **accuracy**: low, medium or high. Triangular functions.
- **speed**: slow, ok or fast. Trapezoidal functions.

These are the rules, where the output variable is difficulty*:

accuracy	speed	difficulty*
low	-	decrease
medium	-	keep
high	fast	increase
high	ok	increase slightly
high	slow	keep
low	slow	decrease strongly

Table 3.7: Fuzzy rules

The fuzzy output is a continuous shift between -1.5 and 1.5 that gets added to the target difficulty.

This approach produces something close to human tutoring behavior, pushing the student to learn more when they are performing well and easing off when they are struggling. We are treating this problem as if it was a PID-kind of problem, bringing in control systems as a solution.

3.3.6. Considerations and limitations

The main aspect of the real learning process to take into account when making these rules is to understand where the sweet spot lies — that is, when real learning is happening. If the user completes everything fast and without mistakes he is most likely not learning anything new, that is when we can increase the difficulty without a doubt. However, for the rest of situations we want to take into account the possibility of the user having very different performances depending on the level of energy, mood or other things out of our reach, so we should not change extremely the difficulty in these cases.

The main thing to promote here is a bit of a challenge, without expecting too much from the user nor too little.

The main limitation for BKT is that we consider knowledge to be timeless, while after a period of time it is possible that a student has forgotten something they already knew. Also, it treats knowledge as a binary variable, you either know something or you do not know it. The forgetting aspect is slightly prevented by the uncertainty (U) heuristic, since it tries to get as many attempts of each KC as possible, and it is proportional, so no KC gets behind after some time.

Moreover, the system treats each KC as independent pieces of knowledge, although in reality these topics have dependencies between each other.

3.4. Application layer

The architecture of the application uses a MAUI cross-platform client, a FastAPI service and MongoDB for the database. The client and server communicate over HTTP using JSON, ensuring that the client has no access to the database and the server does not interact with the UI. This separation is key for the same backend to serve a Windows desktop, Android and iOS app. It is also extremely convenient for the implementation aspect, as modifying code from the back-end or front-end will not break the other side as long as the set of endpoints are respected.

3.4.1. Backend

The backend is implemented in Python. The main reason being that we will be using Ollama to use LLM calls locally. This is used for the dataset generation and for the chatbot itself in the running app. Therefore, for simplicity, I decided to program the whole backend in Python.

The backend application is implemented using the FastAPI library, which allows it to process each HTTP request we define in isolation. The server works with the context provided by each request — through its path or its body — independently from other requests, applying the standard REST property called statelessness. This allows the user state to be safe when restarting the server, but requires all useful state to be stored in the MongoDB database.

3.4.1.1. Routes

The endpoints are divided into four groups, that are specified in different routers of the FastAPI application. The four routers are:

- **users:** owns the user profile, interest and BKT state.
- **problems:** owns the problems lookup.
- **sessions:** owns the recommendation and the attempts results.
- **tutor:** owns the chatbot assistance for solving problems.

Every endpoint declares its request and response bodies with Pydantic classes that are creating using the BaseModel class from FastAPI, as illustrated in Figure 3.1.

```

class AttemptRequest(BaseModel):
    user_id: str
    problem_id: str
    answer: str
    time_spent_seconds: float

class AttemptResponse(BaseModel):
    correct: bool
    correct_answer: str
    updated_bkt: dict[str, float]    # kc -> P(L_kc)
    next_problem_id: str | None

```

Figure 3.1: Pydantic models for the POST `/sessions/{id}/attempt` endpoint.

Each endpoint allows for different verb semantics that hold different properties and are implemented in FastAPI to be used as Python decorators on our router functions. These are the ones that I used:

- **GET**: read only action, used by the backed to fetch problems.
- **POST**: mutates the state and is not idempotent – calling `/session/start` twice will generate two sessions –, it is used for sending an attempt during a session and for receiving the chatbot answer.
- **PUT**: updates one resource and is idempotent – calling PUT `/users/{id}/interests` multiple times with the same body does not change the system after the first one –, it is used for updating the user interests after they redo the questionnaire.

Every endpoint used is summarized in Table 3.8.

Router	Method	Path	Purpose
users	POST	<code>/users</code>	Create user profile
users	GET	<code>/users/{id}</code>	Fetch user profile
users	PUT	<code>/users/{id}/interests</code>	Update questionnaire results
users	GET	<code>/users/{id}/bkt</code>	Fetch current BKT state
problems	GET	<code>/problems</code>	List problems (filter by tag, course, difficulty)
problems	GET	<code>/problems/{id}</code>	Fetch single problem with solution steps
sessions	POST	<code>/sessions/start</code>	Start a new session, returns problem list
sessions	POST	<code>/sessions/{id}/attempt</code>	Submit attempt, update BKT
sessions	GET	<code>/sessions/{id}</code>	Fetch session state and history
tutor	POST	<code>/tutor/chat</code>	Get chatbot reply for a problem

Table 3.8: REST endpoints exposed by the FastAPI backend, grouped by router.

The main benefits of this approach are automatic validation — when there is a missing field or a wrong type, the system returns a 422 error — and automatic documentation accessible at the `/docs` endpoint.

3.4.1.2. Scripts

Python scripts structure:

- app/
 - problems.py -> router that defines http endpoints regarding the problems.
 - sessions.py -> router that defines http endpoints regarding the user session (daily problems, results in problems, updating bkt).
 - tutor.py -> router that defines http endpoints to send chatbot response.
- main.py -> main entry point for fastapi backend, creates backend application.
- ml/
 - bkt/
 - bkt_runtime.py -> implements functions to get the user bkt state, update the bkt state of a user and recommend a set of problems to a user.
 - heuristic.py -> implements a heuristic recommendation model using fixed weights and the user interests to select n problems.
 - chatbot/
 - chatbot.py -> implements the methods that call the chatbot model (DeepSeek-R1) using Ollama and generate the answer to a question of a specific problem.
 - dataset/
 - transformer.py -> implements the generation of the dataset methods during 3 phases that can be executed individually, loads the existing GSM8K dataset and uses KC/tags and taxonomy for consistency.
- scripts/
 - build_dataset.py -> implements main function to generate the dataset using transformer.py modules.

3.4.2. Frontend

The **MVVM pattern**, illustrated in Figure 3.2 is appropriate because MAUI provides a built-in binding system and dependency injection. The framework has embraced modern patterns, including MVVM and MVU, and the **.NET Community Toolkit** reduces boilerplate code for view models. MVVM will allow a clean

separation of views (pages), view models (state and commands) and models (data), simplifying unit testing and facilitating data binding to our recommendation engine.

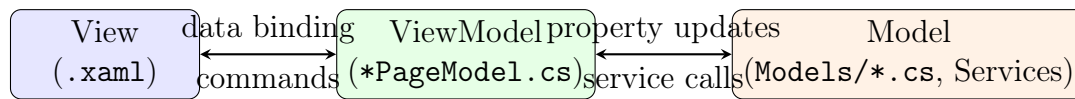


Figure 3.2: MVVM pattern as applied in the MAUI client. Views (XAML pages) bind to ViewModels (PageModel classes), which in turn invoke Services that handle HTTP communication and hold the data Model classes.

4 main pages:

Daily lesson page - presents a curated set of 5-10 problems for the day. The user taps a “Start” button, sees each problem one at a time, submits an answer and receives feedback.

Problem library page - displays all available problems organised by **topic**, **course** and **difficulty level**. A list or grid can show the problem title and tags; tapping an item opens a dedicated problem page.

Profile/interests page - allows students to select their interests (e.g. calculus, probability) from the questionnaire which seed the recommender and show statistics on completed problems.

Settings page - provides different options to modify the configuration of the application, such as turning dark mode on or off.

2 secondary pages:

Problem Detail page - opens the chosen problem from the current session or the problems library, allowing to solve it, ask for hints or ask a question to the chatbot.

Questionnaire page - opens from the profile page and makes the questionnaire to the user, asking him to rate each Knowledge Component, the course he wants to study, the difficulty and his learning objective.

Class structure in .net maui:

```
Models/
|-- ProblemOut.cs
|-- ProblemDetailOut.cs
|-- StartSessionRequest.cs
|-- StartSessionResponse.cs
|-- AttemptRequest.cs
|-- AttemptResponse.cs
|-- TutorChatRequest.cs
|-- TutorChatResponse.cs
|-- UserInterestRequest.cs
'-- UserInterestResponse.cs
Services/
|-- ProblemService.cs
|-- SessionService.cs
|-- TutorService.cs
```

```

'-- UserService.cs
Pages/
|-- MainPage.xaml
|-- ProblemsLibraryPage.xaml
|-- ProblemDetailPage.xaml
|-- ProfilePage.xaml
|-- QuestionnairePage.xaml
'-- SettingsPage.xaml
PageModels/
|-- MainPageModel.cs
|-- ProblemsLibraryPageModel.cs
|-- ProblemDetailPageModel.cs
|-- ProfilePageModel.cs
|-- QuestionnairePageModel.cs
'-- SettingsPageModel.cs
ApiService.cs
App.xaml
AppShell.xaml
GlobalUsings.cs
MauiProgram.cs

```

3.4.3. Navigation example

Figure 3.3 shows one user navigation example for the Windows application in dark mode:

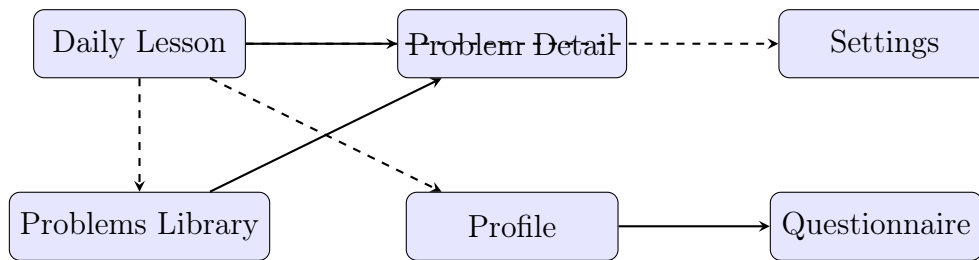


Figure 3.3: Navigation graph between MAUI pages. Solid arrows represent direct, contextual transitions (e.g., tapping a problem opens its detail page). Dashed arrows represent navigation through the app shell tab bar.

The first Page that opens when the user enters is always going to be the Main Page that will contain the session, which is initially empty (Figure 3.4).

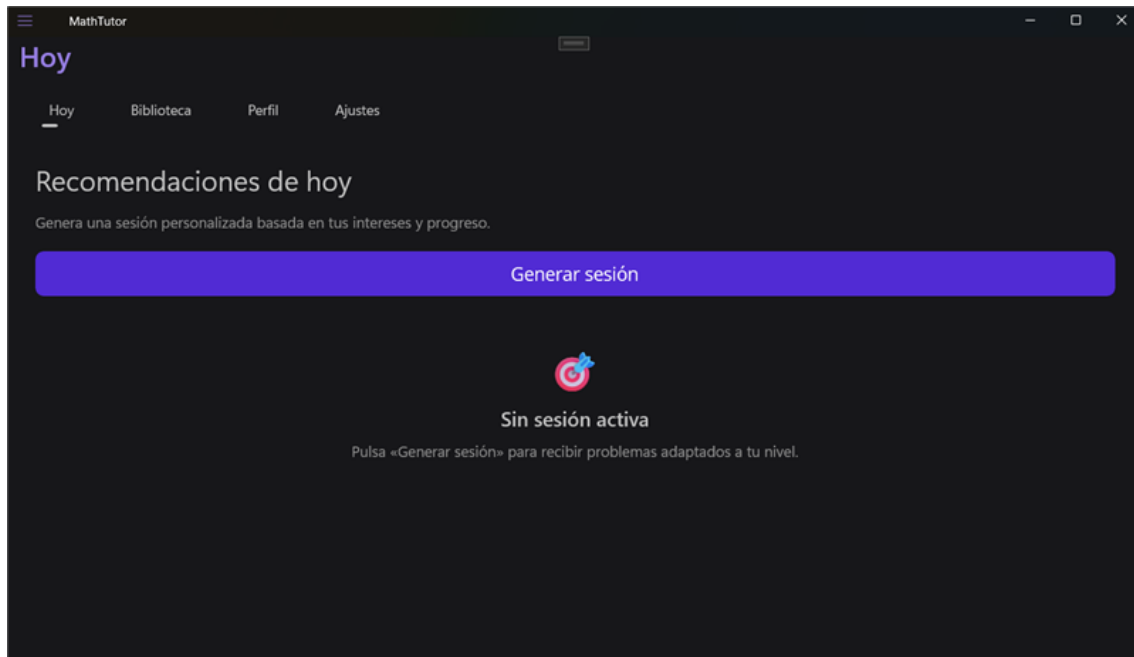


Figure 3.4: Main Page empty

Now, the user has opened the profile page in order to make the questionnaire (Figures 3.5 and 3.6).

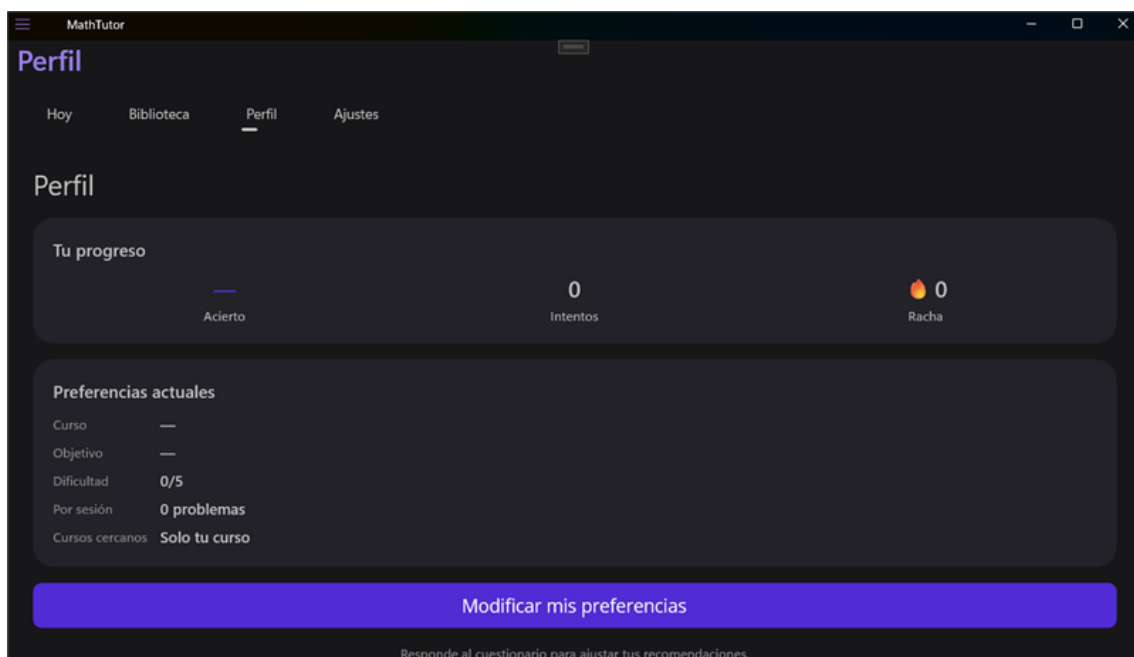


Figure 3.5: Profile Page

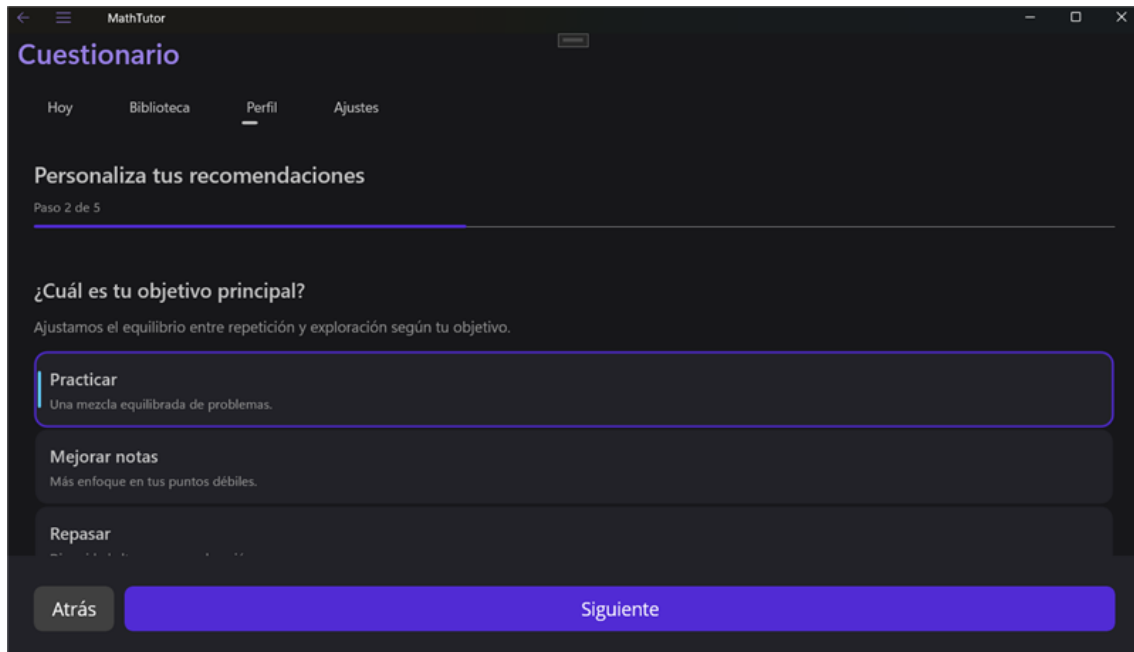


Figure 3.6: Questionnaire Page

Once they are done, the user returns to the Main Page and clicks on the **Generar sesión** button. All the problems appear (Figure 3.7).



Figure 3.7: Main Page full

Afterwards, they navigate to the Problems Library Page in order to find a specific problem (Figure 3.8).

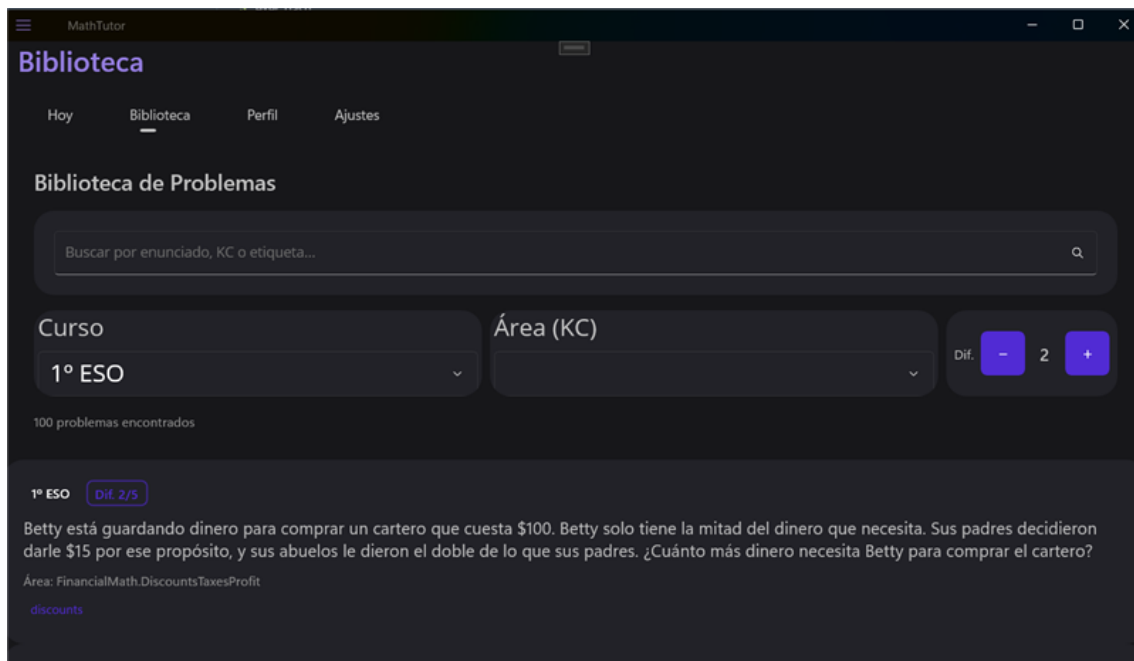


Figure 3.8: Problems Library Page

They have clicked on a problem and it takes them to the Problem Detail Page (Figure 3.9).

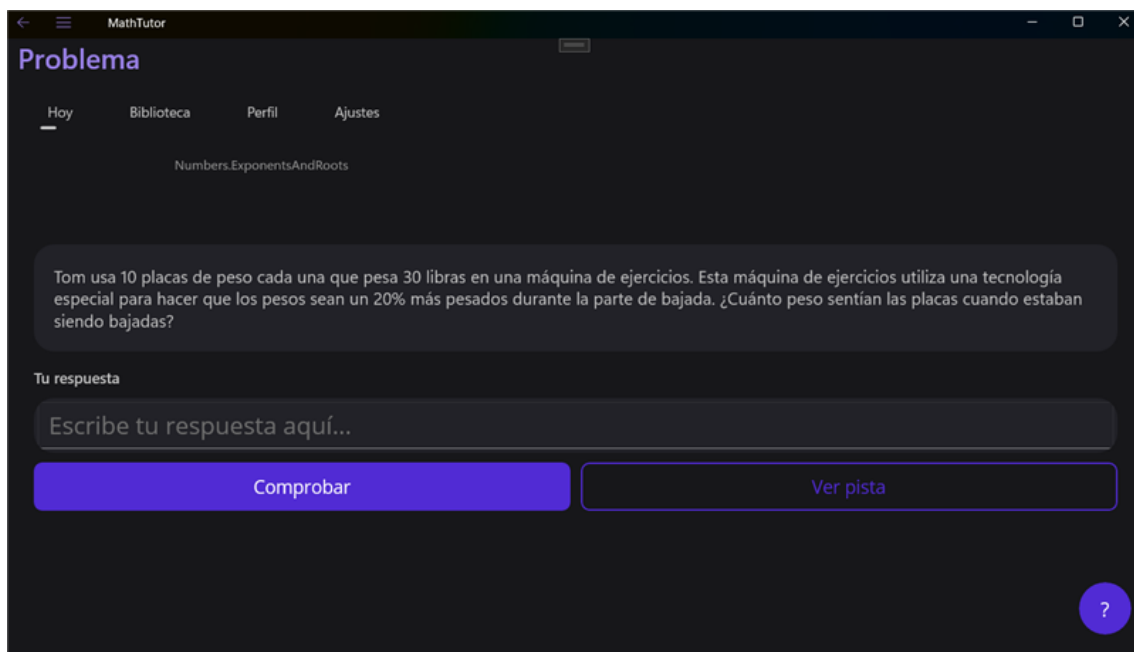


Figure 3.9: Problem Detail Page regular

They can ask for hints or ask a question to the mathstral chatbot (Figures 3.10 and 3.11).

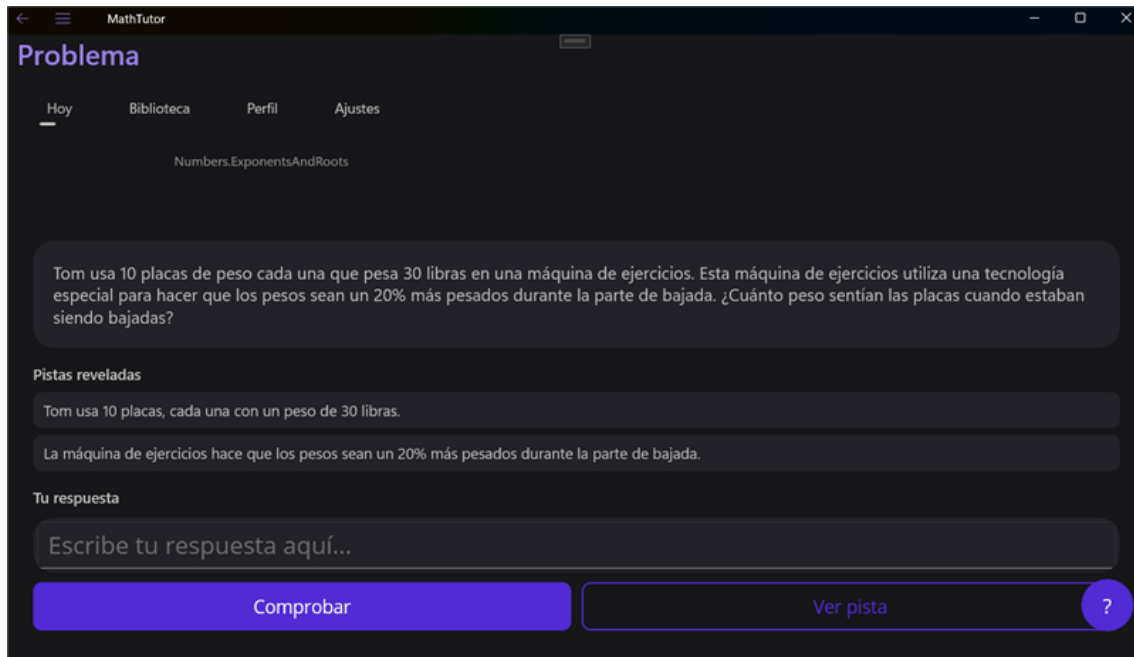


Figure 3.10: Problem Detail Page hints

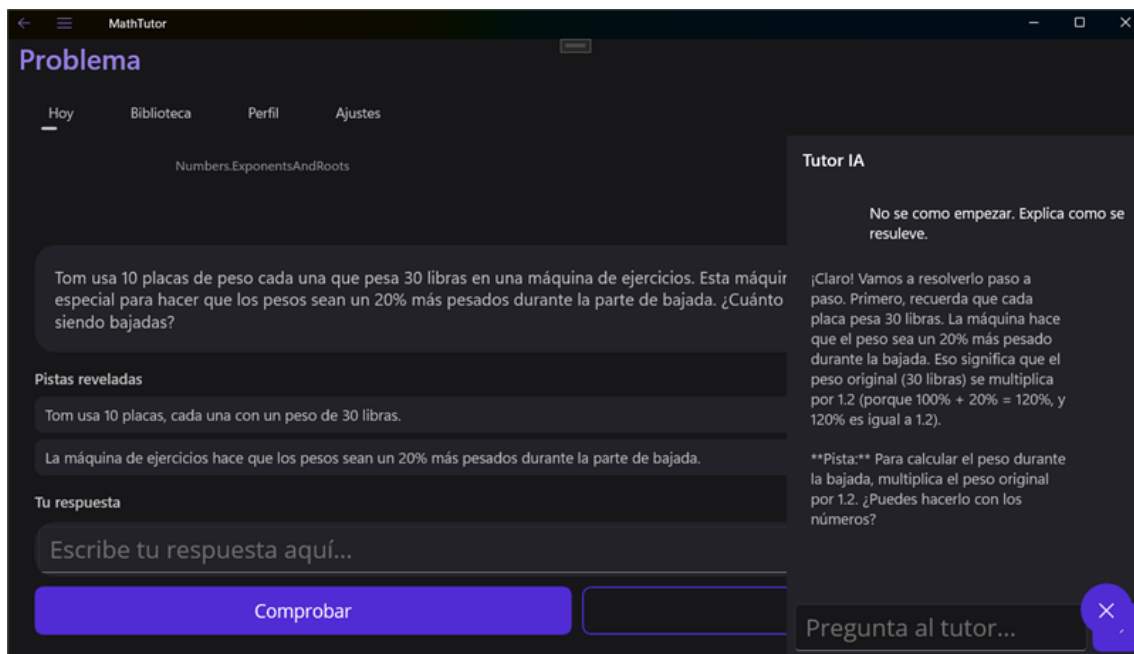


Figure 3.11: Problem Detail Page chatbot

Once they have an answer, they can submit it and check if it is correct (Figure 3.12).

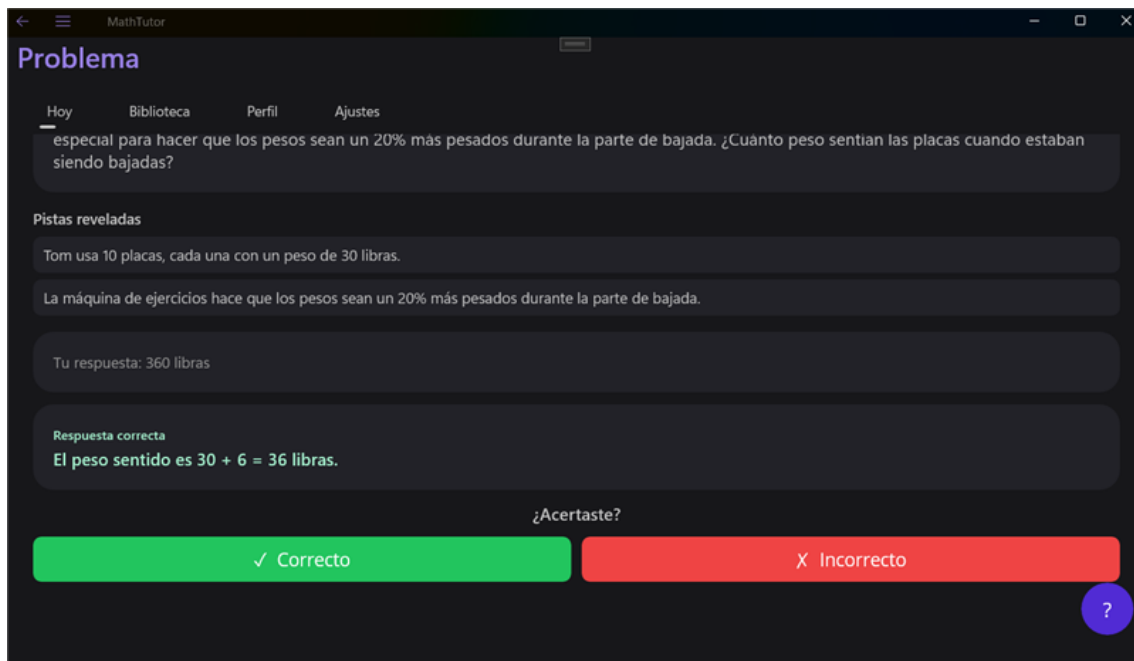


Figure 3.12: Problem Detail Page answer

Finally, they decide to see the Settings Page (Figure 3.13).

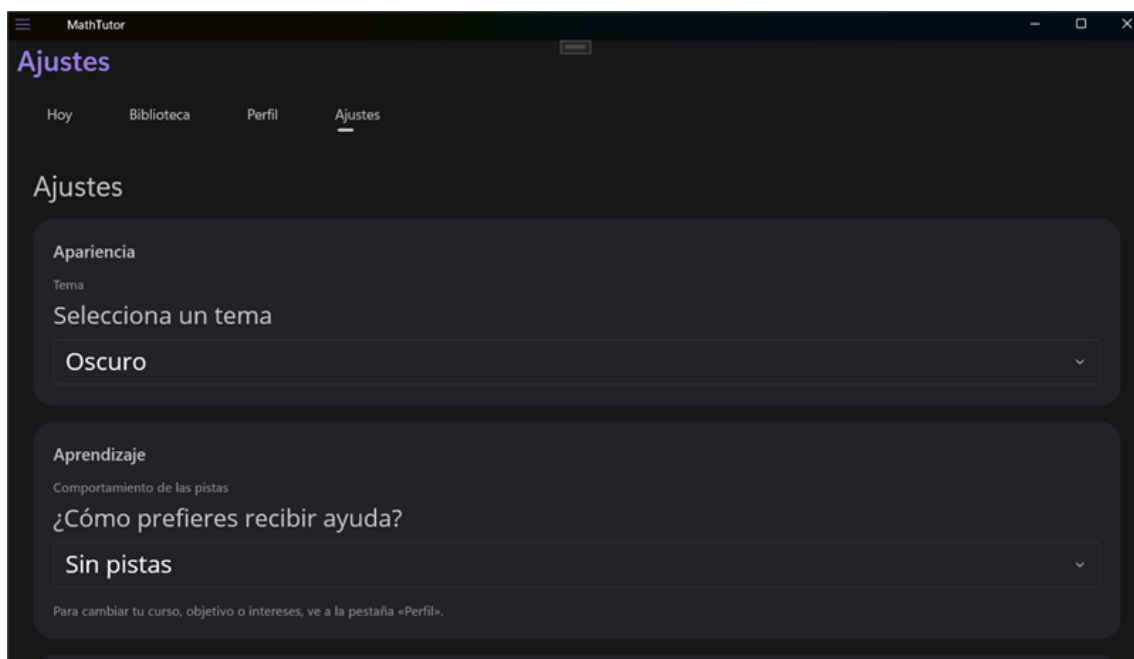


Figure 3.13: Settings Page

Chapter 4

Conclusions and Future Work

This project set out to design a self-driven math tutoring application that combines a curated problem dataset, a personalized recommendation system, and a cross-platform user interface. This chapter compares the objectives set during Chapter 1 against the actual outcomes of each components, and outlines directions for future improvement.

4.1. Conclusions

4.1.1. Dataset generation

799 problems have been processed with 777 problems stored in the database and 707 problems that have passed the validation stage and have been successfully processed for phase C. This results in a 90.99% success rate for the validation process. This table shows the final KC distributions (Table 4.1). It can be seen how the more advanced topics have no representation, this is caused by GSM8K's simplicity and lack of complex content.

Knowledge Component	Count	Proportion (%)
Numbers	375	48.3
FinancialMath	189	24.3
Measurement	113	14.5
Proportionality	70	9.0
Algebra	30	3.9
Sequences	0	0.0
Counting	0	0.0
GeometryPlane	0	0.0
GeometrySolid	0	0.0
Trigonometry	0	0.0
AnalyticGeometry	0	0.0
Functions	0	0.0
Calculus	0	0.0
Statistics	0	0.0
Probability	0	0.0
AppliedModeling	0	0.0

Table 4.1: Distribution of Knowledge Components in the final dataset of 777 problems.

This other table shows the final course distributions (Table 4.2). Both problems from first and second bachillerato are from FinancialMath Knowledge Component. This serves as further proof that the more advanced topics of mathematics are not represented in GSM8K.

Course	Count	Proportion (%)
1° ESO	606	78.0
2° ESO	168	21.6
3° ESO	1	0.1
4° ESO	0	0.0
1° Bach	1	0.1
2° Bach	1	0.1

Table 4.2: Distribution of problems across Spanish secondary-school courses in the final dataset.

Finally, this table contains the difficulty distributions (Table 4.3). These numbers make sense based on the distributions before, as lower courses and simple topics will not demand high difficulties. However, this value depends mostly on the LLM’s criteria.

Difficulty	Count	Proportion (%)
1	463	59.6
2	313	40.3
3	1	0.1
4	0	0.0
5	0	0.0

Table 4.3: Distribution of difficulty levels in the final dataset.

Large Language Models offer a very promising evolution in problem generation. In fact, the base dataset used for this project, GSM8K, is itself a dataset generated using an LLM. The result in this area was satisfactory: the tagging and solution steps fit all the problems nicely. However, this process takes around one minute for each problem across all three phases, which means generating 10,000 problems would require around 170 hours of LLM time. This processing cost is the limiting factor for small LLMs to be fully functional and reliable for this task. Beyond speed, some answers are not completely correct due to the numerous mistakes already present in GSM8K's answers, and the base dataset did not provide a wide enough range of topics, courses and difficulties, which left large content gaps in the final problem dataset. Among the models tested, Mathstral proved to be the most suitable choice, not only for its efficiency but because the simplicity of its model avoided confusion on ambiguous statements and produced complete generations across all three phases, whereas other models such as DeepSeek-R1 suffered from overthinking, which often led to incorrect generations.

4.1.2. Recommendation system

The main limitation of a BKT based recommendation system is that BKT treats knowledge as a binary variable, where a student either knows something or does not. It does not consider forgetting either and Knowledge Components are treated as independent topics, whereas in real life learning from one KC may teach something about a different one. Regardless of these limitations, testing the recommendation system showed good results that adapted to the user's interests and needs. As a user, the hybrid approach for the recommendation system seems to compensate for each component's individual flaws and ends up working as a solid model.

4.1.3. Application

All versions of the application deployed properly, although the testing was done with the Windows version, since I did not have an iOS device and the Android emulator ran extremely slow on my computer. Every feature should theoretically work for every version, as MAUI auto-generates the necessary code for each different platform. The main issue that can be experienced in the application is the Ollama response latency for the chatbot, but this feature should be a last resort as the solution steps generated during the dataset transformation should be enough for a student to know how to solve each problem.

4.1.4. Evaluation against objectives

- **Problem library:** Built a dataset of 707 problems covering 5 out of the 16 Knowledge Components across courses 1^oESO, 2^oESO, 3^oESO, 1^oBach and 2^oBach. Achieved within a limited range.
- **Backend system:** Implemented a FastAPI service with 10 endpoints across 4 routers, backed by MongoDB. Achieved.
- **Cross-platform UI:** Implemented in .NET MAUI. Tested on Windows desktop; Android and iOS builds were not validated due to time constraints. Partially achieved.
- **Recommendation system:** Hybrid BKT + heuristic + softmax sampling implemented. Achieved.

4.2. Future Work

Several directions remain open for extending this work.

4.2.1. Better base dataset

GSM8K, while convenient as a starting point, does not contain problems from the upper Spanish secondary curriculum (Bachillerato). Future work should incorporate a dataset that covers calculus, linear algebra, and statistics at that level. Possible sources include curated problems from Spanish public-exam archives or a fine-tuned generation pipeline trained on Spanish textbook content.

4.2.2. Faster dataset generation

The current three-phase pipeline takes around one minute per problem, dominated by LLM inference. Options to address this include batching, using a smaller distilled model for Phase A (tagging), parallelizing across phases, or running on more powerful GPU hardware.

4.2.3. Engagement and gamification

The current application emphasizes correctness and recommendation quality but does not yet implement progression mechanics. Adding streaks, badges, level-up rewards, or a Hero's Journey-style narrative arc could improve daily-return rates, as evidenced by platforms such as Duolingo.

4.2.4. Recommendation system improvements

The BKT model treats knowledge as binary and assumes Knowledge Components are independent. Future work could explore Deep Knowledge Tracing (DKT) with an LSTM or self-attentive architecture, especially once enough interaction data is

collected to train such a model. A forgetting model (e.g., half-life regression) could also be added to address BKT's main limitation.

4.2.5. Quantitative evaluation

The system has not yet been evaluated against real students. A controlled study comparing learning outcomes between students using the app and a control group would be the natural next step.

4.2.6. Mobile deployment

Although MAUI supports Android and iOS, the application has only been validated on Windows desktop. Future work includes testing and packaging for the mobile platforms.

4.2.7. Closing remarks

From working on this project I have learned that there is much potential for AI in education. It is good to be skeptical, and these new methodologies should be tested before deciding if they are optimal, but I am sure that they can be revolutionary in the area.

Introducción

Introduction to the subject area. This chapter contains the translation of Chapter 1.

Es bien sabido que la educación es uno de los pilares fundamentales de la sociedad. Una generación no puede progresar más que la anterior si no aprende, al menos, las lecciones más valiosas que se enseñaron a la generación previa. Hoy en día, las nuevas tecnologías que utilizan Modelos de Lenguaje Grandes influirán en la forma en la que funciona la educación. A lo largo de este proyecto he tratado de ver qué áreas se pueden ver influidas por esto, y en qué medida.

5.1. Motivación

En la experiencia de muchos estudiantes, lo que se enseña en clase puede quedarse corto respecto a sus expectativas, mientras que en otros casos las lecciones pueden resultarles demasiado exigentes. Independientemente de cuál de estas situaciones aplique, la solución más eficaz es proporcionar al estudiante el apoyo de un profesor particular de matemáticas. Esta solución está respaldada por el artículo de Bloom (1984), donde se analiza cómo los estudiantes que reciben tutoría individualizada combinada con el aprendizaje para el dominio (mastery learning) obtienen un rendimiento dos desviaciones estándar por encima de los estudiantes promedio en un aula tradicional. Sin embargo, esta no es la opción más asequible. Con todos los recientes avances en tecnología e Inteligencia Artificial, debería existir una forma eficaz de sustituir al profesor particular de matemáticas por una aplicación informática.

De hecho, ya existen multitud de programas que buscan resolver este problema, y se estudiarán sus implementaciones en el Capítulo 2. No obstante, no he conseguido encontrar ninguno que se ajuste plenamente al modo en que el sistema educativo español oficial enseña matemáticas, ni que al menos lo apoye de forma autónoma.

Este proyecto adopta un enfoque por capas, donde cada capa se construyó para ser implementada y probada de forma independiente. La capa de datos consiste en construir el conjunto de problemas y montar la infraestructura de base de datos que almacenará las interacciones relevantes del usuario con la aplicación. La capa del sistema de recomendación es la parte inteligente del sistema, encargada de decidir

qué problemas presentar a cada usuario. La capa de aplicación contiene la interfaz multiplataforma con la que interactúa el usuario.

5.2. Objetivos

Los objetivos de este proyecto son:

- Construir una biblioteca de problemas de matemáticas etiquetados por tema, curso y dificultad, adecuados para el currículo de la educación secundaria española.
- Diseñar un sistema backend que exponga estos problemas a través de una API REST.
- Desarrollar una interfaz multiplataforma que enganche a los estudiantes y dé soporte al bucle de aprendizaje basado en sesiones diarias.
- Implementar un sistema de recomendación que personalice la selección de problemas a partir de las interacciones y los intereses declarados del usuario.
- Evaluar el sistema resultante frente a plataformas de tutoría ya existentes.

El alcance de este proyecto se limita al currículo de la educación secundaria española (ESO y Bachillerato). No aborda funciones para docentes (como paneles de control del aula o analíticas), ni implementa el aprendizaje colaborativo multiusuario. La evaluación empírica con estudiantes reales también queda fuera del alcance; la evaluación se restringe a una comparación con las plataformas de tutoría existentes.

5.3. Estructura del documento

El resto de este documento se organiza como sigue. El Capítulo 2 revisa los conjuntos de datos matemáticos existentes, los modelos de lenguaje grandes para razonamiento matemático, las plataformas de tutoría matemática previas y la literatura sobre sistemas de recomendación. El Capítulo 3 describe el diseño e implementación del sistema en sus tres capas: datos, recomendación y aplicación. El Capítulo 4 expone las conclusiones extraídas del trabajo y plantea direcciones para mejoras futuras.

Capítulo 6

Conclusiones y Trabajo Futuro

Conclusions and future lines of work. This chapter contains the translation of Chapter 4.

Este proyecto se planteó diseñar una aplicación autónoma de tutoría matemática que combina un conjunto de problemas curado, un sistema de recomendación personalizado y una interfaz de usuario multiplataforma. Este capítulo compara los objetivos establecidos en el Capítulo 1 con los resultados reales de cada componente, y traza las direcciones para futuras mejoras.

6.1. Conclusiones

6.1.1. Generación del conjunto de datos

Se han procesado 799 problemas, de los cuales 777 se han almacenado en la base de datos y 707 han superado la fase de validación y se han procesado con éxito en la fase C. Esto se traduce en una tasa de éxito del 90,99 % en el proceso de validación. La Tabla 6.1 muestra la distribución final de los Componentes de Conocimiento. Se puede observar cómo los temas más avanzados carecen de representación, lo que se debe a la simplicidad de GSM8K y a su falta de contenidos complejos.

Componente de Conocimiento	Recuento	Proporción (%)
Numbers	375	48,3
FinancialMath	189	24,3
Measurement	113	14,5
Proportionality	70	9,0
Algebra	30	3,9
Sequences	0	0,0
Counting	0	0,0
GeometryPlane	0	0,0
GeometrySolid	0	0,0
Trigonometry	0	0,0
AnalyticGeometry	0	0,0
Functions	0	0,0
Calculus	0	0,0
Statistics	0	0,0
Probability	0	0,0
AppliedModeling	0	0,0

Tabla 6.1: Distribución de los Componentes de Conocimiento en el conjunto de datos final de 777 problemas.

La Tabla 6.2 muestra la distribución final por cursos. Los dos problemas etiquetados como primero y segundo de Bachillerato pertenecen al Componente de Conocimiento FinancialMath. Esto sirve como prueba adicional de que los temas más avanzados de las matemáticas no están representados en GSM8K.

Curso	Recuento	Proporción (%)
1º ESO	606	78,0
2º ESO	168	21,6
3º ESO	1	0,1
4º ESO	0	0,0
1º Bach	1	0,1
2º Bach	1	0,1

Tabla 6.2: Distribución de los problemas por cursos de la educación secundaria española en el conjunto de datos final.

Por último, la Tabla 6.3 contiene la distribución de dificultades. Estos números son coherentes con las distribuciones anteriores, ya que los cursos inferiores y los temas sencillos no exigen dificultades altas. No obstante, este valor depende principalmente del criterio del LLM.

Dificultad	Recuento	Proporción (%)
1	463	59,6
2	313	40,3
3	1	0,1
4	0	0,0
5	0	0,0

Tabla 6.3: Distribución de los niveles de dificultad en el conjunto de datos final.

Los Modelos de Lenguaje Grandes ofrecen una evolución muy prometedora en la generación de problemas. De hecho, el conjunto de datos base utilizado en este proyecto, GSM8K, es a su vez un conjunto de datos generado mediante un LLM. El resultado en este ámbito fue satisfactorio: las etiquetas y los pasos de solución encajan bien con todos los problemas. Sin embargo, este proceso tarda alrededor de un minuto por problema entre las tres fases, lo que significa que generar 10.000 problemas requeriría aproximadamente 170 horas de tiempo de LLM. Este coste de procesamiento es el factor limitante para que los LLM pequeños sean plenamente funcionales y fiables para esta tarea. Más allá de la velocidad, algunas respuestas no son del todo correctas debido a los numerosos errores ya presentes en las respuestas de GSM8K, y el conjunto de datos base no proporcionó un rango suficientemente amplio de temas, cursos y dificultades, lo que dejó grandes huecos de contenido en el conjunto de datos final. Entre los modelos probados, Mathstral resultó ser la opción más adecuada, no solo por su eficiencia sino porque la simplicidad de su modelo evitó confusiones ante enunciados ambiguos y produjo generaciones completas en las tres fases, mientras que otros modelos como DeepSeek-R1 sufrían de exceso de razonamiento, lo que con frecuencia llevaba a generaciones incorrectas.

6.1.2. Sistema de recomendación

La principal limitación de un sistema de recomendación basado en BKT es que BKT trata el conocimiento como una variable binaria, en la que un estudiante o conoce algo o no lo conoce. Tampoco considera el olvido, y los Componentes de Conocimiento se tratan como temas independientes, mientras que en la vida real aprender un KC puede enseñar también algo sobre otro distinto. A pesar de estas limitaciones, las pruebas del sistema de recomendación mostraron buenos resultados, adaptándose a los intereses y necesidades del usuario. Como usuario, el enfoque híbrido del sistema de recomendación parece compensar los defectos individuales de cada componente y acaba funcionando como un modelo sólido.

6.1.3. Aplicación

Todas las versiones de la aplicación se desplegaron correctamente, aunque las pruebas se realizaron con la versión de Windows, ya que no disponía de un dispositivo iOS y el emulador de Android funcionaba extremadamente lento en mi ordenador. Cada funcionalidad debería, en teoría, funcionar en todas las versiones, dado que MAUI genera automáticamente el código necesario para cada plataforma.

El principal inconveniente que se puede experimentar en la aplicación es la latencia de respuesta de Ollama para el chatbot, aunque esta funcionalidad debería ser un último recurso, ya que los pasos de solución generados durante la transformación del conjunto de datos deberían bastar para que un estudiante sepa cómo resolver cada problema.

6.1.4. Evaluación frente a los objetivos

- **Biblioteca de problemas:** Construcción de un conjunto de datos de 707 problemas que cubren 5 de los 16 Componentes de Conocimiento entre los cursos 1º ESO, 2º ESO, 3º ESO, 1º Bach y 2º Bach. Conseguído dentro de un rango limitado.
- **Sistema backend:** Implementación de un servicio FastAPI con 10 endpoints repartidos en 4 routers, respaldado por MongoDB. Conseguído.
- **Interfaz multiplataforma:** Implementada en .NET MAUI. Probada en Windows de escritorio; las versiones de Android e iOS no se validaron por limitaciones de tiempo. Parcialmente conseguido.
- **Sistema de recomendación:** Implementado un enfoque híbrido de BKT + heurísticas + muestreo softmax. Conseguído.

6.2. Trabajo Futuro

Quedan abiertas varias líneas para ampliar este trabajo.

6.2.1. Mejor conjunto de datos base

GSM8K, aunque resulta cómodo como punto de partida, no contiene problemas del tramo superior del currículo de secundaria español (Bachillerato). El trabajo futuro debería incorporar un conjunto de datos que cubra cálculo, álgebra lineal y estadística a ese nivel. Como posibles fuentes se incluyen problemas curados a partir de archivos de exámenes públicos españoles o un pipeline de generación afinado a partir de contenidos de libros de texto españoles.

6.2.2. Generación de datos más rápida

El pipeline actual de tres fases tarda alrededor de un minuto por problema, dominado por la inferencia del LLM. Entre las opciones para abordarlo se incluyen la ejecución por lotes, el uso de un modelo destilado más pequeño para la Fase A (etiquetado), la paralelización entre fases o la ejecución sobre hardware GPU más potente.

6.2.3. Compromiso y gamificación

La aplicación actual prioriza la corrección y la calidad de las recomendaciones, pero todavía no implementa mecánicas de progresión. Añadir rachas, insignias, recompensas por subir de nivel o un arco narrativo al estilo del Viaje del Héroe podría mejorar las tasas de retorno diario, como demuestran plataformas como Duolingo.

6.2.4. Mejoras del sistema de recomendación

El modelo BKT trata el conocimiento como binario y asume que los Componentes de Conocimiento son independientes. El trabajo futuro podría explorar Deep Knowledge Tracing (DKT) con una arquitectura LSTM o auto-atencional, sobre todo una vez que se recopilen suficientes datos de interacción para entrenar un modelo así. También podría añadirse un modelo de olvido (por ejemplo, regresión por vida media) para abordar la principal limitación de BKT.

6.2.5. Evaluación cuantitativa

El sistema aún no se ha evaluado con estudiantes reales. Un estudio controlado que compare los resultados de aprendizaje entre estudiantes que usan la aplicación y un grupo de control sería el siguiente paso natural.

6.2.6. Despliegue móvil

Aunque MAUI da soporte a Android e iOS, la aplicación solo se ha validado en Windows de escritorio. El trabajo futuro incluye probar y empaquetar la aplicación para las plataformas móviles.

6.2.7. Reflexiones finales

Trabajar en este proyecto me ha enseñado que el potencial de la IA en la educación es enorme. Conviene mantener una postura escéptica, y estas nuevas metodologías deberían someterse a pruebas antes de decidir si son óptimas, pero estoy seguro de que pueden ser revolucionarias en este ámbito.

Bibliography

- BLOOM, B. S. The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. *Educational Researcher*, vol. 13(6), páginas 4–16, 1984.
- BRILLIANT.ORG. Brilliant: Learn interactively. <https://brilliant.org/>, 2026. Accessed: 2026-05-22.
- COBBE, K., KOSARAJU, V., BAVARIAN, M., CHEN, M., JUN, H., KAISER, L., PLAPPERT, M., TWOREK, J., HILTON, J., NAKANO, R., HESSE, C. y SCHULMAN, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- CORBETT, A. T. y ANDERSON, J. R. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, vol. 4(4), páginas 253–278, 1995.
- COVINGTON, P., ADAMS, J. y SARGIN, E. Deep neural networks for YouTube recommendations. En *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys '16)*, páginas 191–198. 2016.
- DEEPSEEK-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- FREEMAN, C., KITTREDGE, A., WILSON, H. y PAJAK, B. The Duolingo method for app-based teaching and learning. Duolingo Research Report. https://duolingo-papers.s3.amazonaws.com/reports/Duolingo_whitepaper_duolingo_method_2023.pdf, 2023. Accessed: 2026-05-19.
- GOMEZ-URIBE, C. A. y HUNT, N. The Netflix recommender system: Algorithms, business value, and innovation. *ACM Transactions on Management Information Systems (TMIS)*, vol. 6(4), páginas 1–19, 2015.
- HENDRYCKS, D., BURNS, C., KADAVATH, S., ARORA, A., BASART, S., TANG, E., SONG, D. y STEINHARDT, J. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- KHAN ACADEMY. Khan academy: free online courses, lessons & practice. <https://www.khanacademy.org/>, 2026a. Accessed: 2026-05-22.

KHAN ACADEMY. Khanmigo: Khan academy's ai tutor. <https://www.khanmigo.ai/>, 2026b. Accessed: 2026-05-22.

MISTRAL AI. Mathstral. <https://mistral.ai/news/mathstral>, 2024. Accessed: 2026-05-22.

MITRA, A., KHANPOUR, H., ROSSET, C. y AWADALLAH, A. Orca-Math: Unlocking the potential of SLMs in grade school math. *arXiv preprint arXiv:2402.14830*, 2024.

PIECH, C., SPENCER, J., HUANG, J., GANGULI, S., SAHAMI, M., GUIBAS, L. y SOHL-DICKSTEIN, J. Deep knowledge tracing. *arXiv preprint arXiv:1506.05908*, 2015. Also published in Advances in Neural Information Processing Systems (NeurIPS) 28.

PRODIGY EDUCATION. Prodigy math: Adaptive math learning game. <https://www.prodigygame.com/main-en/>, 2026. Accessed: 2026-05-22.

SANTOSH, E. Decoding Duolingo: How technology and design can shape learning journeys. Medium, GDG VIT. <https://medium.com/gdg-vit/decoding-duolingo-how-technology-design-can-shape-learning-journeys-8a37f48138fc>, 2024. Accessed: 2026-05-19.

SCHAPIRO, A. y TURK-BROWNE, N. Statistical learning. En *Brain Mapping: An Encyclopedic Reference*. Elsevier, 2015.

SCHRAAGEN, J. M., CHIPMAN, S. F. y SHALIN, V. L., editores. *Cognitive Task Analysis*. Psychology Press, 2000.

SHERSTINSKY, A. Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network. *Physica D: Nonlinear Phenomena*, vol. 404, página 132306, 2020.

WEI, C., WANG, B., KIM, J.-J., LIU, G. y CHEN, N. F. CoinMath: Harnessing the power of coding instruction for math LLM. En *Findings of the Association for Computational Linguistics (ACL)*. 2025.

ZHANG, B., LI, C. y FAN, K. MARIO eval: Evaluate your math LLM with your math LLM—a mathematical dataset evaluation toolkit. *arXiv preprint arXiv:2404.13925*, 2024.